

Memoria de Trabajo de Grado

SecureTicket

Solución B2B de reventa de entradas basada en
contratos inteligentes sobre Stellar/Soroban

Versión 6.0 — Mayo de 2026



Pontificia Universidad Javeriana, Bogotá

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas

Director: Rafael Vicente Páez Méndez, PhD

Autores (Grupo 12):

Juan Diego Arias Durán
Santiago Andrés Mesa Niño
Andrés Felipe Manosalva Garzón
Juan Camilo Carvajal Rodríguez

Trabajo de grado presentado como requisito parcial

para optar al título de Ingeniero de Sistemas

Semestre 2026-1

**PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS**

Rector de la Pontificia Universidad Javeriana

Padre Luis Fernando Múnera, S.J.

Decano de la Facultad de Ingeniería

Ing. Diego Alejandro Patiño Guevara, Sc.D.

Directora de la Carrera de Ingeniería de Sistemas

Ing. Andrea del Pilar Rueda Olarte, Ph.D.

Director del Trabajo de Grado

Ing. Rafael Vicente Páez Méndez, Ph.D.

Artículo 23 de la Resolución No. 13 de Junio de 1946

“La Universidad no se hace responsable de los conceptos emitidos por sus alumnos en sus proyectos de grado. Sólo velará porque no se publique nada contrario al dogma y la moral católica y porque los trabajos no contengan ataques o polémicas puramente personales. Antes bien, que se vean en ellos el anhelo de buscar la verdad y la justicia.”

Agradecimientos

Los autores expresan su agradecimiento al Ingeniero Rafael Vicente Páez Méndez, Ph.D., director del trabajo de grado, por su acompañamiento académico, sus revisiones periódicas y la orientación constante durante el desarrollo del proyecto.

Reconocen el aporte del Departamento de Ingeniería de Sistemas de la Pontificia Universidad Javeriana, así como el de los docentes evaluadores que aportaron observaciones a las versiones preliminares del documento.

Finalmente, los autores agradecen a sus familias por el apoyo durante el desarrollo del trabajo de grado.

Índice

	Página
Resumen	VI
Abstract	VII
Glosario	VIII
1. INTRODUCCIÓN	1
2. DESCRIPCIÓN GENERAL	2
2.1. Contexto del Problema	2
2.2. Formulación del Problema	3
2.3. Propuesta de Solución	3
2.4. Justificación de la Solución	3
2.5. Descripción del Proyecto	4
2.5.1. Objetivo General	4
2.5.2. Objetivos Específicos	4
2.5.3. Entregables, Estándares y Justificación	5
3. CONTEXTO DEL PROYECTO	6
3.1. Blockchain, contratos inteligentes y stablecoins	6
3.2. Red Stellar y protocolo de consenso (SCP)	6
3.3. Sostenibilidad y consumo de recursos en perspectiva industrial	8
3.4. Soroban, NFTs y tokens de pago	10
3.5. Análisis del Contexto	10
4. ANÁLISIS DEL PROBLEMA	13
4.1. Requerimientos	13
4.2. Restricciones	16
4.3. Especificación Funcional	17
5. DISEÑO DE LA SOLUCIÓN	19
5.1. Actores, roles y límites de confianza	19
5.2. Modelo del ticket NFT	20
5.2.1. Identificador y separación de datos sensibles	20
5.2.2. Estados y patrón burn/remint	20
5.3. Diseño de contratos inteligentes	21
5.3.1. Contrato Factory	21
5.3.2. Contrato de Evento	22
5.3.3. Contrato NFT	22
5.3.4. Errores tipados y eventos on-chain	23
5.4. Arquitectura distribuida (tres nodos)	24
5.4.1. Nodo web (frontend)	24
5.4.2. Nodo API (backend)	25

5.4.3. Nodo datos (PostgreSQL + indexador)	25
5.5. Coleccionable Stellar Classic	25
6. DESARROLLO DE LA SOLUCIÓN	26
6.1. Implementación de contratos Soroban	26
6.1.1. NFT del boleto: emisión, propiedad y versionado	26
6.1.2. Pagos atómicos vía SAC	26
6.1.3. Redención y control de acceso	27
6.2. Backend Node.js + Stellar SDK	28
6.2.1. API: usuarios, eventos, carrito, checkout y Soroban	29
6.2.2. Indexador: ingesta de eventos on-chain	29
6.2.3. Persistencia: historial, listados y auditoría	30
6.3. Frontend web	31
6.4. Despliegue distribuido	31
6.5. Integración continua	33
6.6. Pruebas	33
7. RESULTADOS	34
7.1. Metodología de medición	34
7.2. Desempeño sobre testnet	34
7.3. Cobertura y resultados de pruebas automatizadas	35
7.4. Evaluación antifraude	35
7.5. Análisis y discusión	36
7.6. Evidencia funcional del prototipo	36
8. CONCLUSIONES	43
8.1. Análisis de Impacto del Proyecto	43
8.1.1. Impacto en ingeniería de sistemas	43
8.1.2. Impacto global, económico, ambiental y social	43
8.2. Conclusiones y Trabajo Futuro	44
Referencias	46
A. ANEXOS	47

Índice de figuras

1.	Ejemplo de rebanadas de quórum (Quorum Slices) para el nodo Bob.	7
2.	Representación matemática e intersección de quóruns en el protocolo SCP. . .	8
3.	Diagrama de casos de uso del sistema (4 actores, 6 casos de uso principales). . .	18
4.	Diagrama de contexto C4 (nivel 1) del sistema SecureTicket.	19
5.	Diagrama de estados del boleto: ACTIVE, USED y CANCELLED, con sus transiciones.	21
6.	Diagrama C4 de contenedores (nivel 2): nodos web, API y datos, y su acoplamiento con Soroban RPC y Horizon.	24
7.	Diagrama de secuencia: compra en reventa atómica (pago al vendedor, comisión a organizador y plataforma, <i>burn</i> de la versión vigente y <i>remint</i> de la nueva, todo en una sola transacción).	27
8.	Diagrama de secuencia: verificación del boleto en puerta (escaneo QR, validación en PostgreSQL y respaldo on-chain).	28
9.	Modelo entidad-relación de la base de datos relacional (esquema ticketing). .	30
10.	Diagrama de despliegue: frontend en Vercel, backend en Railway, base de datos PostgreSQL en Supabase y red Stellar testnet.	32
11.	Consola del usuario administrador: panel desde el que se despliegan los contratos Soroban para eventos <i>offchain</i> y se crean los eventos, vinculando cada evento de la pasarela tradicional con su contrato <i>onchain</i> correspondiente a través del factory	37
12.	Vista “Mis Entradas” del usuario final: los boletos que aún no han sido asegurados en la blockchain se manejan de forma tradicional, por lo que el usuario debe esperar a que el organizador libere el QR de acceso; el botón “Asegurar en Blockchain” permite acuñar el boleto como NFT y liberar el QR de inmediato.	37
13.	Boleto ya asegurado en la blockchain: el QR firmado queda liberado de forma anticipada y la interfaz expone las opciones disponibles sobre el NFT (revender en el mercado P2P, guardar el QR como <i>collectible</i> en la wallet, ver el detalle del contrato).	38
14.	Una vez asegurado el boleto, el usuario puede guardar su QR como NFT <i>collectible</i> dentro de la wallet Freightier; la captura muestra cómo se visualiza el NFT del boleto dentro de la extensión.	38
15.	Listado de un boleto para reventa: el usuario fija el precio en COP, la interfaz muestra el equivalente en XLM a la cotización actual y descompone las comisiones (5 % organizador, 3 % plataforma) junto al monto neto a recibir. . .	39
16.	Vista pública del mercado secundario: así ve un usuario una boleta publicada en la red Stellar disponible para compra P2P, con el precio fijado por el revendedor, el detalle del evento y el botón para ejecutar la compra atómica contra el contrato. . .	40
17.	Ejemplo de cómo el usuario ve en la extensión Freightier el detalle de la transacción Soroban construida por el backend (XDR proxy) y cómo esta solicita su firma con la llave privada local; esta confirmación se solicita tanto al publicar un boleto en reventa como al momento de comprarlo, y las llaves privadas nunca abandonan la extensión.	40

18.	Detalle de la wallet del usuario una vez vinculada su sesión con la extensión Freightier: la interfaz muestra el saldo en XLM y su equivalente en COP a la cotización actual, integrando la información on-chain de la cuenta con la representación monetaria local.	41
19.	Consola del personal de <i>staff</i> (puerta de acceso): los validadores en puerta acceden a un panel reducido cuya única funcionalidad disponible es el “Secure Ticket Scanner”, utilizado para escanear y validar los QR NFT presentados por los asistentes.	41
20.	Vista del <i>Secure Ticket Scanner</i> tal como la observa el personal de <i>staff</i> en puerta: la cámara escanea el QR NFT del asistente y la interfaz reporta el resultado de la verificación contra el contrato <code>ticket_nft_contract</code> , marcando el boleto como redimido en caso de éxito.	42

Índice de tablas

1.	Entregables y estándares aplicados.	5
2.	Comparativa de consumo energético estimado por operación/unidad.	9
3.	Análisis comparativo de soluciones del contexto.	10
4.	Requerimientos funcionales del sistema (resumen).	13
5.	Requerimientos no funcionales del sistema (resumen).	15
6.	Operaciones del contrato de evento.	22
7.	Códigos de error del contrato de evento.	23
8.	Eventos on-chain emitidos por el contrato de evento.	24
9.	Métricas de código fuente y pruebas por contrato.	26
10.	Dependencias principales del backend.	29
11.	Latencia de confirmación y costo nominal por operación sobre testnet.	34
12.	Distribución de pruebas unitarias sobre los contratos Soroban.	35
13.	Escenarios antifraude evaluados sobre el contrato.	35
14.	Anexos del Trabajo de Grado.	47

Resumen

Esta tesis presenta el diseño, la implementación y la validación de una solución B2B de reventa de entradas para eventos de entretenimiento, construida sobre contratos inteligentes Soroban en la red Stellar. El trabajo aborda tres problemas técnicos del mercado secundario: la duplicidad de códigos QR estáticos, la reventa concurrente del mismo activo a varios compradores y la ausencia de trazabilidad pública de la cadena de propiedad.

La propuesta materializa cada boleto como un token no fungible (NFT) identificado por la tupla (`ticket_root_id`, `version`), aplica un patrón *burn/remint* en cada operación de reventa para preservar la unicidad y ejecuta la liquidación de pagos y comisiones de forma atómica dentro de la misma invocación del contrato. El prototipo se despliega como una arquitectura de tres nodos (web, API y datos) sobre la red de prueba de Stellar, con doce eventos desplegados a través de un contrato *factory* (cada evento con su par `event_contract` + `ticket_nft_contract`) y un módulo indexador que mantiene la coherencia entre el ledger y una base de datos relacional.

La evaluación se realiza sobre testnet con métricas reproducibles de latencia de confirmación, costo nominal por operación, cobertura de pruebas unitarias y comportamiento del contrato ante escenarios antifraude. Los resultados permiten contrastar la solución frente a esquemas tradicionales de reventa en términos de tiempo de liquidación, costo por transacción y trazabilidad operacional.

Palabras clave: blockchain, Stellar, Soroban, contratos inteligentes, NFT, reventa de entradas, antifraude, ticketing digital.

Abstract

This thesis presents the design, implementation, and validation of a B2B ticket resale solution for entertainment events, built on Soroban smart contracts on the Stellar network. The work addresses three technical problems of the secondary market: duplication of static QR codes, concurrent resale of the same asset to multiple buyers, and the absence of public traceability of the ownership chain.

The proposal materializes each ticket as a non-fungible token (NFT) identified by the tuple (`ticket_root_id`, `version`), applies a *burn/remint* pattern on each resale operation to preserve uniqueness, and executes payment and commission settlement atomically within the same contract invocation. The prototype is deployed as a three-node architecture (web, API, data) on the Stellar testnet, with twelve events deployed through a factory contract (each event with its `event_contract` + `ticket_nft_contract` pair) and an indexer module that maintains consistency between the ledger and a relational database.

The evaluation is performed on testnet with reproducible metrics of confirmation latency, nominal cost per operation, unit-test coverage, and contract behavior under anti-fraud scenarios. The results enable comparison against traditional resale schemes in terms of settlement time, cost per transaction, and operational traceability.

Keywords: blockchain, Stellar, Soroban, smart contracts, NFT, ticket resale, anti-fraud, digital ticketing.

Glosario

A continuación se definen los términos técnicos utilizados a lo largo del documento. Las definiciones se proporcionan con el nivel de detalle suficiente para que el lector pueda interpretar la memoria sin requerir conocimiento previo sobre tecnologías de registro distribuido.

1. Conceptos Generales de Blockchain y Tokenización

Blockchain (cadena de bloques) : Base de datos descentralizada y replicada formada por bloques de información enlazados criptográficamente de manera inmutable.

Ledger (libro mayor) : Registro digital unificado del estado de la red (balances y transacciones) que los nodos sincronizan mediante un protocolo de consenso.

Hash criptográfico : Algoritmo determinista que transforma un dato de longitud arbitraria en una cadena alfanumérica única de longitud fija, resistente a alteraciones.

Árbol de Merkle : Estructura de datos binaria que resume de forma criptográfica un conjunto de transacciones en un solo hash raíz, permitiendo una verificación ágil.

Doble gasto : Riesgo en sistemas digitales donde un mismo activo financiero o representación de propiedad es transferido a múltiples destinatarios de forma simultánea.

Tokenización : Proceso de digitalización y representación de la propiedad o los derechos de un activo del mundo real mediante tokens transferibles en una blockchain.

Oráculo (*Oracle*) : Canal que conecta y transmite información del mundo exterior en tiempo real hacia los contratos inteligentes en la cadena de bloques.

Web2.5 : Enfoque híbrido que combina la alta velocidad y facilidad de uso de la Web2 convencional con la descentralización y trazabilidad de la Web3.

Consenso : Mecanismo por el cual los nodos de la red acuerdan el contenido de un nuevo bloque. Stellar utiliza el *Stellar Consensus Protocol* (SCP), distinto a Proof of Work y Proof of Stake.

Finalidad (*finality*) : Propiedad por la cual una transacción confirmada no puede ser revertida. En Stellar la finalidad se alcanza en el cierre del *ledger*, en un intervalo de 3 a 5 segundos.

Mainnet / Testnet : Redes operativas de Stellar. La *mainnet* es la red de producción con activos de valor real; la *testnet* es la red de pruebas con activos sin valor económico, utilizada en este trabajo.

2. Protocolo e Infraestructura de la Red Stellar

Stellar : Red descentralizada y pública optimizada para la emisión de activos digitales y el envío transfronterizo de valor con alta velocidad y bajo costo.

Soroban : Plataforma de contratos inteligentes de Stellar, integrada desde el protocolo 20. Los contratos se escriben en Rust y se compilan al formato WebAssembly.

Quórum / *Quorum slice* : Subconjunto de nodos cuya colaboración basta para que un nodo acepte un valor como acordado. Un quórum contiene al menos un pedazo de cada uno de sus miembros.

Stroop : Unidad mínima de XLM. Un XLM equivale a 10^7 stroops. Se utiliza como unidad de almacenamiento por precisión entera; los costos en esta memoria se reportan convertidos a COP.

FBA (*Federated Byzantine Agreement*) : Modelo de consenso abierto donde cada nodo validador elige de forma autónoma a sus propios socios de confianza en lugar de depender de una membresía fija.

SCP (*Stellar Consensus Protocol*) : Implementación del modelo FBA que prioriza la consistencia y seguridad del estado del ledger sobre la disponibilidad del servicio ante fallos.

Rebanada de quórum (*Quorum slice*) : Grupo de nodos en el que un validador deposita su confianza. La superposición de estas rebanadas en la red determina el quórum global.

Anclaje (*Anchor*) : Entidad regulada de la red Stellar que actúa como puente financiero para procesar depósitos fiat y emitir tokens equivalentes en el ledger.

AMM (*Automated Market Maker*) : Creador de mercado automatizado en Stellar que permite intercambiar activos sin intermediarios mediante fondos de liquidez gobernados por algoritmo.

Horizon : Capa de servicios HTTP que proporciona a las aplicaciones Web2 una API interactiva con las funcionalidades clásicas de Stellar.

Soroban RPC : Interfaz que permite a los desarrolladores enviar transacciones, simular ejecuciones y consultar el estado operativo de los contratos inteligentes.

XLM (*Lumen*) : Criptoactivo nativo de Stellar utilizado para cubrir tarifas de transacción en el ledger e impedir el spam en la red.

Stroop : Unidad de medida mínima de la red Stellar, equivalente a la diezmillonésima parte (10^{-7}) de un Lumen (XLM).

Trustline (línea de confianza) : Registro operacional en el ledger de Stellar mediante el cual una cuenta aprueba explícitamente recibir y poseer un token emitido por un tercero.

Friendbot : Herramienta automatizada en la red de pruebas (Testnet) de Stellar para acreditar fondos de XLM ficticios a cuentas en desarrollo.

3. Contratos Inteligentes Soroban y Estándares

Contrato inteligente Soroban : Programa de ejecución lógica inmutable y determinista que corre en la máquina virtual segura de Soroban y está escrito en Rust.

WASM (WebAssembly) : Formato de instrucciones binarias rápido y multiplataforma al que se compilan los contratos inteligentes para asegurar su reproducción unívoca en la red.

SAC (*Stellar Asset Contract*) : Contrato nativo de Stellar que encapsula los tokens clásicos del protocolo para que puedan usarse de forma homogénea en Soroban.

SEP-41 : Propuesta de estándar oficial de Stellar que unifica la interfaz y firmas de las funciones típicas para tokens en el entorno de Soroban.

Almacenamiento de Soroban : Esquema de tres niveles de persistencia de datos con costes de renta según su vida útil: *Instance* (metadatos), *Persistent* (datos restaurables) y *Temporary* (datos efímeros).

NFT (*Non-Fungible Token*) : Token digital único, identificable y no divisible que representa de forma verificable la propiedad de un activo en la red.

Acuñación (*Mint*) : Acción de crear y registrar por primera vez un token criptográfico dentro del ledger de la cadena de bloques.

Quema (*Burn*) : Acción de retirar de forma irreversible un token en circulación, marcándolo como inactivo o destruido en el storage del contrato.

Patrón *Burn/Remint* : Técnica transaccional atómica que invalida la versión actual de un boleto y crea de inmediato una nueva versión del mismo a nombre de su comprador.

Clawback (recuperación) : Característica transaccional que faculta al administrador de un activo para retirar y reasignar un token sin requerir la firma de su titular actual.

Atomicidad : Propiedad de una transacción en la que todas las operaciones agrupadas se validan conjuntamente o no se ejecuta ninguna (*todo o nada*).

Eventos de Soroban : Registros de log efímeros y económicos que emiten los contratos inteligentes en el ledger y que son leídos por aplicaciones externas de monitoreo.

XDR (*External Data Representation*) : Estándar de serialización de datos binario empleado por Stellar para formatear transacciones de forma consistente entre plataformas.

4. Arquitectura y Componentes de la Solución (Stellar Tickets)

Freighter : Extensión web que funciona como billetera digital segura y no custodial, permitiendo a los usuarios firmar localmente transacciones en la red Stellar.

Indexador (*Indexer*) : Servicio backend permanente encargado de consultar el RPC de Soroban para reflejar las actualizaciones de la blockchain en la base de datos PostgreSQL.

Proxy XDR : Componente del backend del sistema que modela transacciones crudas en XDR para que sean firmadas por el cliente sin custodiar sus llaves privadas.

Contrato fábrica (*Factory contract*) : Contrato inteligente central encargado de desplegar e inicializar de forma programática un contrato independiente para cada evento

creado.

Redención operativa : Escaneo de un código QR y comprobación de su versión en una base de datos relacional para dar acceso rápido al recinto del evento.

Redención on-chain : Llamada al método del contrato inteligente en Soroban para asentar e invalidar de forma inmutable el boleto utilizado en la blockchain.

5. Siglas, Estándares y Abreviaturas

API : *Application Programming Interface* (Interfaz de Programación de Aplicaciones).

B2B : *Business to Business* (Relaciones comerciales de Empresa a Empresa).

E2E : *End to End* (Pruebas de flujo completo de Extremo a Extremo).

JWT : *JSON Web Token* (Método estándar de transmisión segura de identidad mediante tokens de corta duración).

KYC / AML : *Know Your Customer* (Conozca a su Cliente) y *Anti-Money Laundering* (Prevención de Lavado de Activos).

ORM : *Object-Relational Mapping* (Mapeo de objetos a bases de datos relacionales; en el proyecto se emplea Prisma).

RPC : *Remote Procedure Call* (Llamada a Procedimiento Remoto para interacción con nodos blockchain).

SPA : *Single Page Application* (Aplicación web que carga en una única página interactiva sin recarga de navegador).

TPS : *Transactions per Second* (Capacidad de procesamiento de transacciones por segundo en una red).

USDC / USDC_SIM : Token estable emitido por Circle con paridad 1 : 1 respecto al dólar estadounidense; en el proyecto se simula su comportamiento comercial.

UVT : Unidad de Valor Tributario. Unidad de referencia monetaria en Colombia utilizada para establecer límites de contribuciones parafiscales de espectáculos.

1. INTRODUCCIÓN

La digitalización del mercado de boletería ha facilitado la compra y la transferencia de entradas, pero también ha incrementado los riesgos asociados a la reventa fraudulenta: duplicidad de códigos de acceso, venta concurrente del mismo activo a varios compradores y opacidad del historial de propiedad. Estos vectores afectan la confianza del comprador, la liquidación efectiva entre los actores del mercado y la trazabilidad operacional del organizador.

Este documento presenta el diseño, la implementación y la validación de **SecureTicket**, una solución B2B de reventa de entradas basada en contratos inteligentes Soroban sobre la red Stellar. La propuesta materializa cada boleto como un *token* no fungible (NFT) con versionado, aplica un patrón *burn/remint* en cada reventa para preservar la unicidad y ejecuta la liquidación de pagos y comisiones de forma atómica dentro de la misma invocación del contrato.

El documento se organiza en nueve secciones. La sección 2 presenta la descripción general del proyecto, incluyendo problema, propuesta de solución, objetivos y entregables. La sección 3 expone el contexto técnico y el análisis de soluciones comparables. La sección 4 detalla los requerimientos, restricciones y la especificación funcional del sistema. La sección 5 desarrolla la arquitectura y el diseño detallado de los contratos y los nodos de la solución. La sección 6 describe el proceso de implementación, despliegue e integración continua. La sección 7 consolida los resultados experimentales sobre la red de prueba (*testnet*). La sección 8 analiza el impacto del trabajo y presenta las conclusiones y el trabajo futuro. La sección 9 lista las referencias bibliográficas. Finalmente, los anexos referencian el Plan de Gestión del Proyecto, la Especificación de Requisitos de Software y el documento de Diseño y Propuesta del Trabajo de Grado, entregados como artefactos independientes.

2. DESCRIPCIÓN GENERAL

2.1. Contexto del Problema

El mercado secundario de boletería digital para espectáculos de entretenimiento y deportes opera actualmente sobre profundas carencias de información y fallos de seguridad que afectan de forma sistemática a los consumidores, organizadores y artistas. Aunque la transición tecnológica desde las entradas impresas tradicionales hacia los boletos con código QR dinámico o estático optimizó los canales de distribución primaria, también introdujo nuevos vectores de explotación fraudulenta que las bases de datos relacionales centralizadas y las pasarelas de pago tradicionales no han logrado erradicar por construcción.

A nivel global, la industria de la reventa de boletos es un sector con una valoración estimada de entre 3 000 y más de 30 000 millones de dólares anuales [1]. Paralelo a este crecimiento, los esquemas de fraude transaccional y falsificación de entradas generan pérdidas globales que superan los 1 600 millones de dólares al año, afectando de manera directa a cerca de 5 millones de usuarios que adquieren boletos fraudulentos en canales no oficiales [2]. Esta problemática se sustenta principalmente en tres puntos de fricción técnica y operativa:

1. **Duplicación y Reventa Concurrente:** Un código QR estático o una representación en PDF de un boleto constituye una cadena de datos binaria susceptible de ser clonada indefinidamente a costo marginal cero. Si un revendedor comercializa el mismo archivo a múltiples compradores de forma paralela en mercados informales, el sistema de validación centralizado de la ticketera local solo detectará la duplicidad en el acceso físico al evento, denegando el ingreso al resto de los compradores de buena fe.
2. **Asimetría de Liquidación Financiera:** Las pasarelas de pago Web2 tradicionales liquidan los fondos de transacciones internacionales en ventanas temporales que oscilan entre 72 y 120 horas hábiles, mientras que el boleto digital es transferido al comprador de forma inmediata. Esta brecha temporal permite a revendedores maliciosos vender entradas, recibir el capital y posteriormente realizar contracargos bancarios o reclamaciones de fraude, dejando al comprador final sin el activo y sin los fondos.
3. **Ausencia de Trazabilidad Pública de la Cadena de Propiedad:** Al no existir un registro público y transparente que certifique el historial de transferencias del boleto desde su acuñación primaria por el organizador hasta el consumidor final, resulta imposible verificar la legitimidad de un boleto adquirido en el mercado secundario antes de la validación en puerta.

A nivel regional y local en Colombia, la debilidad técnica de las plataformas tradicionales ha propiciado escenarios de fraude y cartelización de alta visibilidad. Un caso representativo ocurrió en el año 2020, cuando la Superintendencia de Industria y Comercio (SIC) sancionó a la Federación Colombiana de Fútbol (FCF) y a la tiquetera oficial Ticketshop con una multa superior a los 4,6 millones de dólares por la estructuración de una red de reventa masiva y desvío de más de 42 000 entradas para las eliminatorias de la Copa Mundial de la FIFA Rusia 2018, reportando sobrecostos especulativos de hasta el 350 % sobre la tarifa nominal [3]. De igual forma, en conciertos multitudinarios celebrados en el país durante el año 2022, las autoridades locales reportaron miles de casos de estafas masivas por duplicidad de códigos QR distribuidos a través de plataformas informales de reventa P2P, evidenciando que las tiqueteras

convencionales no disponen de mecanismos para invalidar copias previas al instante de una transferencia.

2.2. Formulación del Problema

El problema de investigación que aborda este trabajo se formula bajo la siguiente interrogante: *¿cómo diseñar e implementar un middleware B2B de arquitectura Web2.5 que se acople a las plataformas de boletería digital existentes para garantizar, por construcción y de forma atómica, la unicidad y autenticidad del boleto en el mercado secundario, permitiendo regular de manera flexible la especulación y asegurar la liquidación inmediata de pagos y comisiones, sin forzar a los usuarios a interactuar directamente con la complejidad técnica de la Web3?*

Este planteamiento da a proponer tres condiciones de diseño: la inhabilitación física de códigos anteriores debe ocurrir de forma atómica con la confirmación de pago; el sistema debe operar como un servicio de bajo costo que se acople a la infraestructura existente en lugar de reemplazarla; y la propiedad del ticket debe validarse on-chain con el fin de eliminar la necesidad de un intermediario de custodia financiera.

2.3. Propuesta de Solución

La propuesta consiste en una plataforma de boletería digital híbrida bajo una arquitectura Web 2.5 estructurada como un middleware B2B. El diseño del sistema no busca competir de manera directa con las ticketeras consolidadas del mercado, lo cual generaría resistencia comercial e incrementaría la fricción técnica para el usuario final que suele tener desconfianza o desconocimiento frente a los criptoactivos y billeteras Web3. En su lugar, el prototipo se concibe como un servicio auxiliar de bajo costo transaccional con el que las ticketeras existentes se acoplan mediante APIs REST.

Las plataformas de tickets tradicionales preservan su estructura Web2 (catálogo, reserva de asientos físicos, frontend tradicional y autenticación JWT). Sin embargo, delegan en el middleware la liquidación transaccional y la transferencia de propiedad on-chain sobre la red Stellar. Cada boleto se modela como un NFT en el entorno de Soroban [4] identificado por la tupla (`ticket_root_id`, `version`). Durante la compra en reventa, el middleware construye un XDR que ejecuta de forma atómica el patrón *burn/remint* (invalidando la versión v y emitiendo la versión $v + 1$ al comprador) y la distribución del pago en tokens estables USDC [5].

La especulación de precios se maneja como una política moldeable. En lugar de imponer reglas que limiten la flexibilidad del negocio, la restricción de precio máximo de reventa y el porcentaje de comisiones se establecen como opciones configurables en el contrato inteligente al momento de su inicialización, supeditadas al acuerdo comercial exclusivo o a las políticas operativas de la ticketera organizadora del evento.

2.4. Justificación de la Solución

La selección Stellar como blockchain y su motor de contratos inteligentes *Soroban* como infraestructura de soporte se justifica por sus propiedades orientadas al rápido procesamiento de transacciones financieras de manera económica:

- **Latencia de Finalidad Determinista:** El algoritmo de consenso SCP de Stellar [6] garantiza la finalidad absoluta de una transacción en un rango de 3 a 5 segundos. Esta velocidad es coherente con los tiempos de respuesta exigidos en pasarelas de pago y evita demoras en los accesos físicos del aforo.
- **Bajo Costo Transaccional:** La comisión base en Stellar es de 100 stroops ($\approx 0,02$ COP a la tasa de referencia), lo que permite procesar miles de transacciones de reventa y distribución de regalías sin afectar el valor comercial de la entrada.
- **Atomicidad Nativa y USDC via SAC:** A través del Stellar Asset Contract (SAC) [7], las operaciones financieras de pago en USDC y las operaciones de estado del boleto (quema de la versión anterior y re-minteo del nuevo NFT) se ejecutan dentro del mismo marco de invocación transaccional de Soroban. Si alguna de las transferencias o validaciones de propiedad falla, la transacción se revierte en su totalidad, eliminando la necesidad de construir sistemas complejos de depósito en garantía (*escrow*) que son vulnerables a fallos y retención indebida de fondos.
- **Arquitectura de Proyección Relacional:** El acoplamiento Web2.5 se apoya en un indexador asíncrono que proyecta el historial on-chain en una base de datos relacional PostgreSQL. Esto permite que las aplicaciones web de las tiqueteras realicen consultas con latencias de milisegundos, ofreciendo una experiencia convencional de alta velocidad de lectura sin obligar al usuario a lidiar directamente con las llamadas de solo lectura a la blockchain.
- **Alta Capacidad de Procesamiento (TPS):** El protocolo Stellar soporta una tasa nominal de procesamiento de entre 1 000 y 3 000 transacciones por segundo (TPS) según la complejidad de las operaciones [8]. Aunque esta cifra es inferior a la capacidad teórica de redes centralizadas tradicionales como Visa, que reporta picos de hasta 65 000 TPS con promedios de procesamiento reales de entre 1 700 y 4 000 TPS [9], resulta órdenes de magnitud superior a otras redes blockchain públicas como Ethereum 1.0 (que oscila entre 15 y 30 TPS) y es suficiente para absorber las demandas de alta concurrencia transaccional en eventos de boletería masivos.

2.5. Descripción del Proyecto

2.5.1. Objetivo General

Desarrollar una solución B2B de reventa de tickets, basada en la tecnología de contratos inteligentes de Stellar, que garantice la unicidad y autenticidad de cada activo digital, reduciendo al mínimo el fraude y la especulación en la reventa de entradas para eventos de entretenimiento.

2.5.2. Objetivos Específicos

- Crear un sistema de reventa B2B de tickets basado en contratos inteligentes y NFTs que aseguren autenticidad, originalidad y trazabilidad.
- Desarrollar un contrato inteligente que genere tickets como NFTs, permita la transferencia de propiedad y automatice la distribución de comisiones.

- Implementar un prototipo que simule el proceso completo de reventa en la red de prueba de Stellar, desde la venta hasta la compra.
- Gestionar el funcionamiento de la herramienta a través de una implementación web que permita la simulación y visualización del modelo y de su integración con la cadena de bloques.

2.5.3. Entregables, Estándares y Justificación

La Tabla 1 consolida los entregables del trabajo de grado, los estándares aplicados como marco de referencia y la justificación de cada uno.

Tabla 1: Entregables y estándares aplicados.

Entregable	Estándar de referencia	Justificación
Memoria de Trabajo de Grado (este documento)	Formato institucional PUJ	Documento autocontenido que sintetiza el trabajo realizado.
Plan de Gestión del Proyecto (SPMP)	Plantilla institucional	Trazabilidad de la planeación, hitos, riesgos y presupuesto.
Especificación de Requisitos (SRS)	Plantilla institucional	Definición detallada de RF, RNF, casos de uso y criterios de aceptación.
Especificación del Diseño (SDD) y Propuesta de TG	Modelo C4 + secuencias UML	Vistas de contexto, contenedores, componentes y despliegue.
Suite de tres contratos Soroban + 58 pruebas	Rust + <code>soroban-sdk</code> 23	Materialización del modelo NFT con versionado, <i>burn/remint</i> y NFT SEP-41.
Backend Node.js + indexador	Express + Prisma + PostgreSQL	API REST, XDR proxy <i>stateless</i> y Sincronización on-chain/off-chain.
Frontend web (React/Vite)	SPA responsive + Freightier	Flujo E2E para usuarios finales y operadores.
Integración continua	GitHub Actions	Compilación y pruebas automatizadas en cada <i>push</i> .

Los artefactos de gestión, requisitos, diseño y verificación se organizan de manera reproducible y auditable a partir del código fuente público y de la documentación entregada como anexos.

3. CONTEXTO DEL PROYECTO

3.1. Blockchain, contratos inteligentes y stablecoins

Una cadena de bloques es una estructura de datos distribuida, cronológica y enlazada criptográficamente, en la que un conjunto de nodos independientes mantiene una réplica del registro y aplica un protocolo de consenso para acordar el orden y la validez de las transacciones sin requerir una autoridad central. Para evitar ataques de denegación de servicio y de doble gasto, las blockchains públicas tradicionales se apoyan en dos paradigmas principales de consenso:

- **Proof of Work (PoW):** Introducido por Bitcoin [10], requiere que los nodos validadores (mineros) resuelvan complejos acertijos criptográficos utilizando potencia de cómputo. El primero en resolver el acertijo tiene el derecho de proponer el siguiente bloque, asegurando la red a través del gasto intensivo de energía.
- **Proof of Stake (PoS):** Alternativa que reemplaza el consumo energético por el compromiso financiero. Los validadores bloquean una cantidad de criptomonedas nativas (colateral) en la red para tener la probabilidad de proponer nuevos bloques, penalizando económicamente los comportamientos maliciosos.

Sobre esta infraestructura descentralizada, los contratos inteligentes son programas informáticos con estado persistente y ejecución determinística cuyas transiciones se ejecutan bajo el mismo conjunto de reglas en cada nodo. En el ámbito del ticketing, el contrato inteligente actúa como una bóveda lógica auto-ejutable que regula la propiedad, transfiere el activo y distribuye las regalías al organizador de forma automática al recibir el pago.

Sin embargo, las criptomonedas nativas presentan una volatilidad de precio que introduce fricción comercial en el e-commerce cotidiano. Para mitigar esta problemática, el ecosistema blockchain introdujo las *Stablecoins* (monedas estables), tokens criptográficos cuyo valor se encuentra anclado en paridad 1 : 1 a una moneda fiduciaria de curso legal (como el dólar estadounidense). En este trabajo se adopta USDC (USD Coin) como token de pago, un criptoactivo respaldado en su totalidad por reservas de efectivo y bonos del Tesoro de EE. UU. [5], lo que permite que el comprador de un boleto pague una tarifa fija en dinero estable mientras disfruta de la programabilidad y liquidación atómica on-chain.

3.2. Red Stellar y protocolo de consenso (SCP)

Stellar es una blockchain pública orientada a la transferencia rápida de valor digital y a la emisión de activos tokenizados. A diferencia de las redes tradicionales basadas en PoW o PoS, la red Stellar opera bajo el *Stellar Consensus Protocol* (SCP), una implementación del modelo de Acuerdo Bizantino Federado (FBA, *Federated Byzantine Agreement*) [6]. El FBA establece un paradigma descentralizado en el cual los participantes del sistema no son preseleccionados por una autoridad central, sino que cada nodo declara individualmente sus propios círculos de confianza denominados **rebanadas de quórum** (Quorum Slices).

Un quórum global en SCP se define como un conjunto de nodos que contiene al menos una rebanada de quórum para cada uno de sus miembros. Para ilustrar este concepto, la documentación oficial de Stellar propone un modelo de toma de decisiones locales mediante un ejemplo compuesto por cuatro participantes: Bob, Cal, Cam y Deb. En este escenario, Bob

desea acordar el estado de una transacción y define sus condiciones de confianza de la siguiente manera:

1. Confía de forma conjunta en la opinión de Cal y Cam.
2. Confía de forma independiente en la opinión de Deb.

Esto significa que Bob posee dos rebanadas de quórum válidas: el conjunto $\{\text{Bob}, \text{Cal}, \text{Cam}\}$ y el conjunto $\{\text{Bob}, \text{Deb}\}$. Para que Bob valide un bloque, es necesario que él y todos los miembros de al menos una de sus rebanadas estén de acuerdo. Este flujo de confianza subjetiva y local se representa gráficamente en la Figura 1.

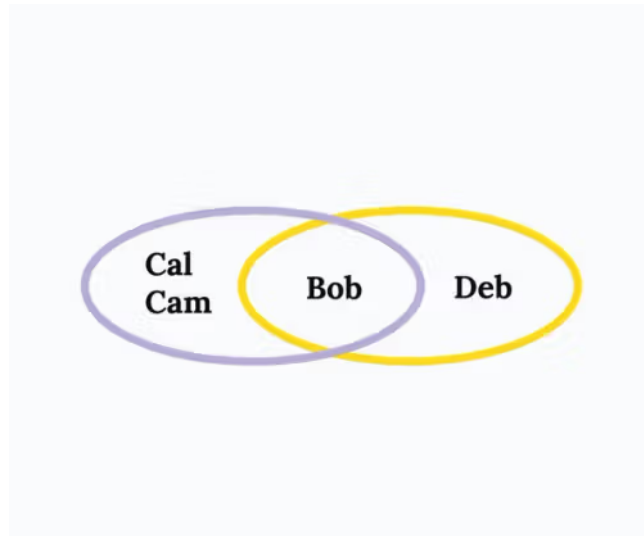
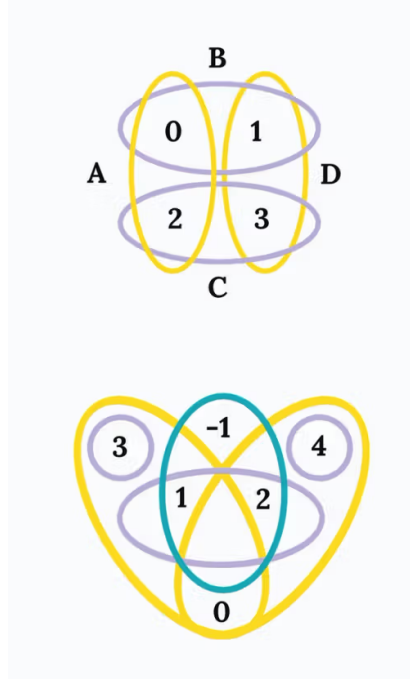
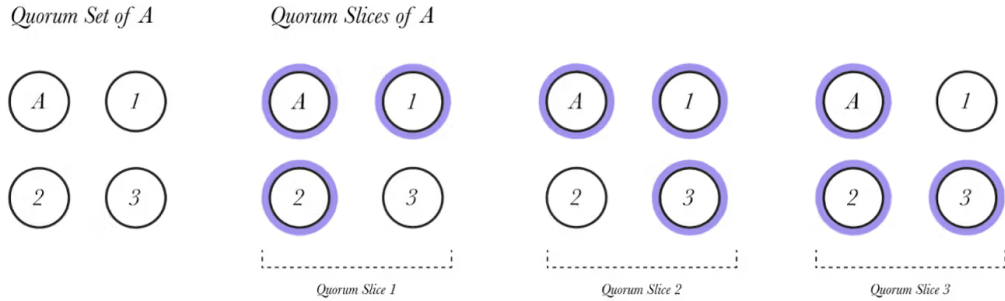


Figura 1: Ejemplo de rebanadas de quórum (Quorum Slices) para el nodo Bob.

El SCP garantiza que, a través de la superposición de estas rebanadas de confianza locales entre múltiples participantes en la red, se formen quóruns globales interconectados. La propiedad fundamental del SCP es la **intersección de quóruns**: si todo par de quóruns de la red comparte al menos un nodo correcto (no defectuoso ni malicioso), el sistema mantendrá seguridad absoluta (*safety*) impidiendo la bifurcación del registro histórico (doble gasto), incluso en presencia de nodos bizantinos. El SCP prioriza en todo momento la consistencia del registro sobre la disponibilidad temporal (*liveness*), deteniendo la red en caso de que no exista consenso en lugar de escribir transacciones inconsistentes. En la Figura 2 se muestra la representación matemática del solapamiento de estas rebanadas y nodos.



(a) Intersección de quóruns lógicos.



(b) Configuración de consenso entre nodos.

Figura 2: Representación matemática e intersección de quóruns en el protocolo SCP.

3.3. Sostenibilidad y consumo de recursos en perspectiva industrial

La adopción de tecnologías descentralizadas se enfrenta de manera recurrente a debates de carácter ecológico. La minería clásica de Bitcoin (PoW) consume aproximadamente 150 TWh de energía eléctrica al año [11], una huella energética equiparable al consumo anual de países enteros como Argentina o Ucrania, lo cual se traduce en una emisión significativa de gases de efecto invernadero si la matriz energética depende de combustibles fósiles.

En contraste, el mecanismo SCP de Stellar no involucra minería por cómputo intensivo ni colateralización forzosa de validadores, limitándose al intercambio asíncrono de mensajes firmados criptográficamente. Una evaluación del impacto ecológico comisionada por la Stellar Development Foundation en conjunto con la consultora PwC US bajo el Marco de Evaluación

de Sostenibilidad de Blockchain (BSF, *Blockchain Sustainability Framework*) determinó los siguientes valores operativos para la red [12]:

- **Consumo de energía por transacción:** Aproximadamente 0,173 Wh (equivalente a 0,000173 kWh).
- **Emisión de carbono por transacción:** Aproximadamente 0,062 gramos de CO_2 equivalente (gCO_2e).
- **Consumo total del ecosistema:** La energía eléctrica anual total requerida por el conjunto completo de validadores activos de Stellar equivale al consumo energético de un único servidor web convencional de gama media.

Para poner estas cifras en perspectiva dentro de la infraestructura tecnológica y económica moderna, la Tabla 2 contrasta el consumo de energía unitario estimado de diferentes actividades y sectores contra el procesamiento de una transacción en Stellar.

Tabla 2: Comparativa de consumo energético estimado por operación/unidad.

Actividad / Sector	Consumo de energía	Equivalencia en transacciones Stellar
Transacción en Stellar (SCP)	0,173 Wh (0,000173 kWh)	1
Consulta simple en buscador tradicional	$\approx 0,30$ Wh	$\approx 1,7$
Consulta simple a modelo de IA (LLM)	$\approx 3,00$ – $9,00$ Wh	≈ 17 – 52 [13]
Transacción clásica de tarjeta de crédito	$\approx 1,50$ – $2,50$ Wh	≈ 8 – 14
Producción de 1 kg de cemento (Construcción)	$\approx 1\,300$ – $1\,500$ Wh	$\approx 7\,500$ – $8\,600$
Transacción promedio de Bitcoin (PoW)	≈ 800 – $1\,200$ kWh	$\approx 4\,600$ millones– $6\,900$ millones [11]

A nivel global, mientras que los centros de datos (impulsados en gran parte por el crecimiento exponencial del entrenamiento e inferencia de modelos de inteligencia artificial y la computación en la nube) proyectan consumir más de 1 000 TWh para el año 2026 [13] y el sector de la construcción tradicional aporta cerca del 37 % de las emisiones de carbono mundiales, la red Stellar se establece como una de las infraestructuras de liquidación financiera más sostenibles de la actualidad. La integración de este middleware Web2.5 no penaliza la huella de carbono de los eventos masivos, sino que la optimiza en comparación con el despliegue de contratos inteligentes sobre cadenas heredadas basadas en PoW o redes centralizadas de alta fricción física (transporte y tickets en papel).

3.4. Soroban, NFTs y tokens de pago

Soroban es la plataforma de contratos inteligentes de Stellar. Los contratos se escriben en Rust contra el crate `soroban-sdk` (versión 23 en este trabajo) y se compilan al target `wasm32v1-none`. El modelo de ejecución es determinístico, sin acceso a fuentes no reproducibles (relojes locales, aleatoriedad no derivada del ledger, llamadas al sistema operativo).

Soroban gestiona tres tipos de almacenamiento: **Persistent**, **Temporary** e **Instance**, cada uno con un costo de *rent* distinto. La emisión de eventos se realiza mediante `events().publish()`: cada evento contiene *topics* indexables y un *body* serializable, y se almacena en el meta del ledger, lo que permite a los indexadores externos consumirlo sin afectar el costo de *rent* del contrato.

El estándar NFT en Soroban define una representación de activos no fungibles con identificador único, propietario tipo **Address** y reglas de transferencia controladas por el contrato emisor. En este trabajo se extiende este estándar con un esquema de versionado: cada boleto se identifica por la tupla (`ticket_root_id`, `version`), donde `version` se incrementa en cada operación de reventa mediante un patrón *burn/remint*.

El activo nativo de la red, XLM, y cualquier activo emitido en Stellar se exponen como tokens compatibles con la interfaz estándar de Soroban a través del *Stellar Asset Contract* (SAC). Esto permite que las operaciones de pago de la compra de reventa se ejecuten dentro del mismo flujo de invocación del contrato del evento, sin transacciones separadas para el pago y la transferencia de propiedad.

3.5. Análisis del Contexto

El análisis comparativo de soluciones del mercado y de la literatura técnica permite identificar tres categorías de antecedentes: (i) plataformas tradicionales de reventa basadas en QR estáticos y conciliación bancaria; (ii) soluciones NFT desplegadas sobre Ethereum 1.0 o sus capas 2; y (iii) propuestas académicas de *ticketing* distribuido sin liquidación atómica. La Tabla 3 contrasta los principales parámetros técnicos de cada categoría contra la propuesta de este trabajo.

Tabla 3: Análisis comparativo de soluciones del contexto.

Parámetro	Tradicional	NFT en ETH 1.0	Académica	Propuesta
Finalidad	72–120 h	1–6 min	Variable	3–5 s
Costo por op.	1,5–3,5 % + fija	USD 5–50 (gas)	N/A	≈ 0,02 COP
Capacidad	—	15–30 TPS	—	1 000–3 000 TPS
Atomicidad pago + propiedad	No	Parcial	No	Sí
Trazabilidad pública	No	Sí	Parcial	Sí
Unicidad por construcción	No	Sí	Variable	Sí (<i>burn/remint</i>)

A continuación se describe cada uno de los parámetros evaluados en la Tabla 3 y el razonamiento que apoyan a cada categoría:

- **Finalidad (tiempo de liquidación):** Las plataformas tradicionales dependen de la conciliación interbancaria, cuyo ciclo de liquidación oscila entre 72 y 120 horas hábiles; durante ese intervalo, el vendedor no dispone de los fondos y el comprador carece de garantía irrevocable sobre la transferencia. Las soluciones NFT sobre Ethereum 1.0 reducen esta ventana al tiempo de confirmación de bloque (entre 1 y 6 minutos, dependiendo de la congestión de la red y del número de confirmaciones exigidas), mientras que las propuestas académicas revisadas no definen un mecanismo de liquidación determinístico, por lo que su finalidad se clasifica como variable. La propuesta de este trabajo, al operar sobre el protocolo SCP de Stellar, alcanza una finalidad absoluta en un rango de 3 a 5 segundos, un orden de magnitud comparable al tiempo de respuesta percibido por el usuario en una pasarela de pago convencional.
- **Costo por operación:** En el modelo tradicional, las comisiones de procesamiento de pago combinan un porcentaje variable (entre el 1,5 % y el 3,5 % del monto) con una tarifa fija por transacción, costos que se trasladan al consumidor o al organizador del evento. En Ethereum 1.0, el costo de gas por la ejecución de un contrato inteligente de transferencia de NFT puede situarse entre 5 y 50 dólares estadounidenses en períodos de alta demanda, lo que resulta prohibitivo para transacciones de boletería de bajo valor nominal. La propuesta aprovecha la estructura de tarifas de Stellar, donde la comisión base es de 100 stroops (equivalente a aproximadamente 0,02 pesos colombianos a la tasa de referencia), un valor que no introduce fricción económica apreciable.
- **Capacidad de procesamiento (TPS):** Las plataformas tradicionales no reportan una métrica de TPS comparable, dado que la capacidad depende de la infraestructura centralizada del operador y no de un protocolo de consenso distribuido. Ethereum 1.0 ofrece una capacidad nominal de entre 15 y 30 transacciones por segundo, insuficiente para absorber picos de demanda simultánea en la venta masiva de boletería. Stellar soporta entre 1000 y 3000 TPS según la complejidad de las operaciones, una cifra que cubre holgadamente los escenarios de concurrencia propios de eventos de entretenimiento a gran escala.
- **Atomicidad pago + propiedad:** Este parámetro evalúa si la transferencia de propiedad del boleto y la liquidación del pago ocurren dentro de una misma transacción indivisible. Las plataformas tradicionales separan ambos procesos en flujos distintos (la entrega del código QR es inmediata, mientras que el pago se liquida días después), lo que genera la ventana de vulnerabilidad descrita en la formulación del problema. Ethereum 1.0 ofrece atomicidad parcial: si bien es posible agrupar operaciones en un contrato, la práctica habitual recurre a patrones de *escrow* o *commit-reveal* que introducen pasos intermedios y puntos de fallo. La propuesta de este trabajo ejecuta el patrón *burn/remint* y la distribución de pagos y comisiones en una sola invocación transaccional de Soroban, garantizando que todas las operaciones se confirmen o se reviertan de forma conjunta.
- **Trazabilidad pública:** Las tiqueteras tradicionales almacenan el historial de transferencias en bases de datos privadas, inaccesibles para el comprador o para terceros auditores. Las soluciones sobre Ethereum y la propuesta de este trabajo registran cada transfe-

rencia en un ledger público e inmutable, lo que permite a cualquier interesado verificar la cadena de custodia del boleto desde su emisión original. Las propuestas académicas revisadas implementan trazabilidad parcial, limitada a los participantes del protocolo y sin exposición pública verificable.

- **Unicidad por construcción:** Un código QR estático en una plataforma tradicional es una cadena de bytes replicable a costo marginal cero; la detección de duplicados solo ocurre en el momento de la validación en puerta. Las soluciones NFT sobre Ethereum garantizan unicidad mediante el estándar ERC-721, donde cada token posee un identificador irrepetible dentro del contrato. La propuesta extiende este principio con el esquema de versionado (`ticket_root_id`, `version`): cada operación de reventa destruye la versión vigente del NFT y acuña una nueva con versión incrementada, de modo que no puedan coexistir dos representaciones válidas del mismo boleto en ningún instante del tiempo.

Como referencia normativa local, la Ley 1493 de 2011 regula los espectáculos públicos de las artes escénicas en Colombia y establece una contribución parafiscal del 10 % sobre la boletería cuyo precio individual sea igual o superior a tres UVT. El modelo propuesto registra el precio efectivo on-chain en cada operación, lo que habilita el cálculo determinístico de la contribución mediante una agregación sobre el historial indexado. Este mapeo se utiliza como referencia académica y no constituye declaración de cumplimiento regulatorio productivo.

4. ANÁLISIS DEL PROBLEMA

4.1. Requerimientos

El sistema se especifica mediante veinte requerimientos funcionales (RF) y veinte requerimientos no funcionales (RNF), organizados en cinco módulos: M1 gestión de eventos y tickets, M2 emisión NFT, M3 mercado secundario, M4 configuración y políticas, M5 usuarios y autenticación. La especificación detallada con criterios de aceptación y trazabilidad a casos de uso se entrega como anexo (SRS). Las Tablas 4 y 5 resumen el conjunto completo.

Tabla 4: Requerimientos funcionales del sistema (resumen).

Id.	Descripción	Módulo
RF-01	Comprar boletos para un evento mediante interfaz clara.	M1, M5
RF-02	Validar que la compra sea autorizada (reglas, permisos, control de usuario).	M1, M4, M5
RF-03	Generar identificador único para cada boleto, vinculable al NFT.	M1, M2
RF-04	Permitir compra con flujo sencillo.	M1, M5
RF-05	Autenticar al usuario antes de comprar.	M5
RF-06	Registrar propietario inicial y final del boleto (trazabilidad).	M1, M2, M3, M5
RF-07	Listar boleto para reventa.	M3, M4, M5
RF-08	Verificar que el usuario sea dueño antes de revender.	M2, M3, M5
RF-09	Ejecutar compra en reventa de forma atómica.	M2, M3, M4, M5
RF-10	Evitar que el boleto se venda más de una vez.	M1, M2, M3
RF-11	Ejecutar proceso de <i>burn</i> del boleto vendido.	M2, M3
RF-12	Generar nuevo token para nuevo propietario.	M2, M3, M5
RF-13	Mantener historial de transferencias.	M1, M2, M3, M5
RF-14	Consultar estado de boletos adquiridos.	M1, M2, M5
RF-15	Verificación rápida del boleto.	M1, M2
RF-16	Solo el propietario puede modificar el precio de reventa.	M3, M4, M5
RF-17	Integrarse con contratos inteligentes Soroban.	M2, M3, M4
RF-18	Modificar precio del boleto listado para reventa.	M3, M4, M5
RF-19	Consultar disponibilidad de boletos.	M1
RF-20	Registrar cada transacción realizada.	M1, M2, M3, M5

Los veinte requerimientos funcionales se organizan a lo largo de cinco módulos que cubren el ciclo de vida completo del boleto digital, desde la adquisición inicial hasta la auditoría posterior al evento. A continuación se describe la lógica de cada módulo y su aporte al objetivo del sistema:

- **M1 – Gestión de eventos y tickets.** Agrupa las operaciones de catálogo, compra primaria y consulta de disponibilidad (RF-01, RF-03, RF-04, RF-10, RF-14, RF-19). Estos requerimientos garantizan que el usuario pueda explorar eventos, adquirir entradas y consultar su inventario dentro de una interfaz convencional, sin necesidad de interactuar de forma directa con la capa blockchain.

- **M2 – Emisión NFT.** Define la acuñación del boleto como activo no fungible en Soroban y las operaciones de *burn/remint* asociadas al cambio de propiedad (RF-03, RF-06, RF-08, RF-11, RF-12, RF-13, RF-15, RF-17, RF-20). Cada boleto se identifica por la tupla (`ticket_root_id`, `version`), cuya integridad se verifica on-chain mediante `require_auth()` sobre la dirección del propietario [4].
- **M3 – Mercado secundario.** Cubre el flujo de reventa segura: listado del boleto, verificación de propiedad, ejecución atómica de la compra y distribución de comisiones (RF-07, RF-08, RF-09, RF-10, RF-11, RF-12, RF-13, RF-16, RF-18, RF-20). La atomicidad de la transacción constituye el requisito central de este módulo, dado que elimina la ventana de vulnerabilidad al doble gasto descrita en la formulación del problema.
- **M4 – Configuración y políticas.** Establece las reglas de negocio parametrizables del contrato: precio máximo de reventa, porcentaje de comisión del organizador y tarifa de plataforma (RF-02, RF-04, RF-07, RF-09, RF-16, RF-17, RF-18). Estos parámetros se fijan en la inicialización del contrato y son modificables exclusivamente por la dirección del organizador.
- **M5 – Usuarios y autenticación.** Gestiona la identidad del usuario en las dos capas del sistema: autenticación JWT en la capa Web2 y firma criptográfica mediante la extensión Freighter en la capa Web3 (RF-01, RF-02, RF-04, RF-05, RF-06, RF-07, RF-08, RF-09, RF-12, RF-13, RF-14, RF-16, RF-18, RF-20). La separación de responsabilidades entre ambas capas asegura que las llaves privadas del usuario nunca abandonen el entorno local de la extensión del navegador.

Los veinte requerimientos no funcionales (RNF) se clasifican conforme al modelo de calidad de producto de la norma ISO/IEC 25010 [14], cubriendo seis atributos de calidad: desempeño, seguridad, usabilidad, disponibilidad, mantenibilidad e integración. La Tabla 5 resume el conjunto completo.

Tabla 5: Requerimientos no funcionales del sistema (resumen).

Id.	Descripción
RNF-01	Procesar la compra de boletos en menos de 5 segundos.
RNF-02	Soportar al menos 100 usuarios concurrentes.
RNF-03	Tener disponibilidad mínima del 99,5 % mensual.
RNF-04	Bloquear transacciones tras 3 intentos fallidos consecutivos.
RNF-05	Cifrar comunicaciones mediante TLS 1.2 o superior.
RNF-06	Soportar al menos 2 400 transacciones diarias.
RNF-07	Completar el proceso de compra en máximo 5 pasos.
RNF-08	Permitir aprendizaje del usuario en menos de 15 minutos.
RNF-09	Mantener una tasa de fallos menor al 0,5 %.
RNF-10	Garantizar atomicidad en el 100 % de las transacciones.
RNF-11	Tener cobertura de pruebas mínima del 70 %.
RNF-12	Ser compatible con navegadores web modernos.
RNF-13	Integrarse con contratos Soroban mediante API REST.
RNF-14	Almacenar <i>logs</i> por mínimo 12 meses.
RNF-15	Cerrar sesión tras 30 minutos de inactividad.
RNF-16	No superar 5 horas de inactividad mensual.
RNF-17	Verificar boletos en menos de 5 segundos.
RNF-18	Soportar varios eventos simultáneos.
RNF-19	Recuperarse de fallos en menos de 20 minutos.
RNF-20	Conservar historial de boletos por mínimo 5 años.

La justificación de los umbrales definidos en cada requerimiento no funcional se organiza a continuación por atributo de calidad:

- **Desempeño (RNF-01, RNF-02, RNF-06, RNF-17, RNF-18).** El umbral de 5 segundos para la compra y la verificación del boleto se deriva de la finalidad del protocolo SCP, que garantiza la confirmación de una transacción en un rango de 3 a 5 segundos [6]. La concurrencia mínima de 100 usuarios simultáneos y las 2 400 transacciones diarias se establecen como línea base para un evento de aforo mediano (aproximadamente 5 000 asistentes), considerando que la mayor densidad transaccional se concentra en la apertura

de la venta y en los 30 minutos previos al ingreso.

- **Seguridad (RNF-04, RNF-05, RNF-10, RNF-15).** El bloqueo tras tres intentos fallidos mitiga ataques de fuerza bruta sobre las credenciales de sesión. El cifrado mediante TLS 1.2 o superior se alinea con las recomendaciones de la OWASP para la protección de datos en tránsito [15]. La atomicidad total de las transacciones (RNF-10) se garantiza por el modelo de ejecución de Soroban, donde una invocación que falle en cualquiera de sus sub-operaciones revierte el estado completo del ledger sin efectos parciales [4]. El cierre de sesión por inactividad limita la exposición de la sesión JWT en equipos compartidos.
- **Usabilidad (RNF-07, RNF-08, RNF-12).** La restricción de un máximo de cinco pasos para completar una compra responde al principio de diseño de que cada paso adicional en un flujo de pago incrementa la tasa de abandono del usuario. El tiempo de aprendizaje de 15 minutos sitúa la curva de adopción del sistema en un rango aceptable para usuarios sin experiencia previa en criptoactivos o billeteras Web3. La compatibilidad con navegadores modernos asegura que la extensión Freightier y las llamadas a Soroban RPC se ejecuten sin restricciones.
- **Disponibilidad (RNF-03, RNF-16, RNF-19).** La disponibilidad del 99,5 % mensual admite un máximo acumulado de aproximadamente 3,6 horas de inactividad al mes, lo que se complementa con el techo explícito de 5 horas de RNF-16. El tiempo de recuperación de 20 minutos ante fallos establece un objetivo de *Mean Time To Recovery* (MTTR) coherente con despliegues en plataformas de infraestructura como servicio (Railway, Vercel).
- **Mantenibilidad (RNF-09, RNF-11, RNF-14, RNF-20).** La tasa de fallos inferior al 0,5 % y la cobertura de pruebas mínima del 70 % se establecen como indicadores de fiabilidad del código. El almacenamiento de logs durante 12 meses permite la trazabilidad operativa de incidentes, mientras que la conservación del historial de boletos por 5 años responde a la necesidad de auditoría y a la persistencia on-chain de los eventos indexados.
- **Integración (RNF-13).** La comunicación entre la capa Web2 y la capa Web3 se canaliza a través de una API REST que actúa como proxy XDR *stateless*. El backend construye la transacción Soroban, la serializa en formato XDR y la devuelve al frontend para que el usuario la firme localmente con Freightier, siguiendo el patrón de integración documentado en la guía oficial de desarrollo de aplicaciones sobre Stellar [16].

4.2. Restricciones

Las restricciones del prototipo se agrupan en cuatro categorías:

- **De alcance.** El sistema se limita al mercado secundario de entradas para eventos de entretenimiento. Quedan fuera la integración con pasarelas de pago fiat reales, los procesos KYC, el despliegue en *mainnet* y el acoplamiento con plataformas comerciales de *ticketing*.
- **Técnicas.** El despliegue se realiza sobre la red de prueba (*testnet*) de Stellar. La compra del comprador final se firma exclusivamente desde la extensión Freightier en el navegador.

Las operaciones clásicas (`change_trust`, `payment`) se enrutan por Horizon; las invocaciones de contrato, por Soroban RPC. El indexador no se ejecuta en entornos *serverless* dado que requiere proceso de fondo persistente.

- **Normativas.** La Ley 1493 de 2011 se incluye como referencia académica para la fiscalización digital de espectáculos públicos en Colombia.
- **Operativas.** La política de comisiones del prototipo se fija en 5 % organizador y 3 % plataforma; estos parámetros son configurables por contrato pero no se modifican durante la evaluación. La ventana de retención del método `getEvents` del RPC público obliga al indexador a poll-ear cada 5 segundos.

4.3. Especificación Funcional

El sistema involucra cuatro actores principales, alineados con los casos de uso del SDD. Cada rol queda acotado y, cuando interviene on-chain, queda verificado por el contrato mediante `require_auth()` sobre la dirección Stellar correspondiente:

- **Administrador de la Plataforma.** Configura eventos en el panel administrativo y dispara el despliegue de contratos de evento mediante el `factory_contract`.
- **Organizador del evento (ticketera B2B).** Única dirección autorizada para invocar `crear_boleto`, registrar verificadores e invalidar boletos en un contrato de evento. Es receptor de las comisiones de organizador.
- **Usuario.** Comprador y vendedor del mercado primario y secundario. El propietario actual firma `listar_boleto` y `cancelar_venta`; el comprador firma `comprar_boleto` desde Freightier.
- **Personal del Evento (verificador en puerta).** Rol STAFF o ADMIN registrado por el organizador. Ejecuta la redención en puerta a través del endpoint `POST /api/admin/scan`, con respaldo on-chain mediante `redimir_boleto`.

El diagrama de casos de uso (Figura 3) sintetiza la interacción de los cuatro actores con las funcionalidades principales del sistema, agrupadas por módulo. La especificación detallada de cada caso de uso (CU-01 Crear evento y desplegar contrato, CU-02 Compra primaria, CU-03 Listar para reventa, CU-04 Compra en reventa atómica, CU-05 Verificar en puerta, CU-06 Invalidar boleto) se documenta en el anexo SDD.

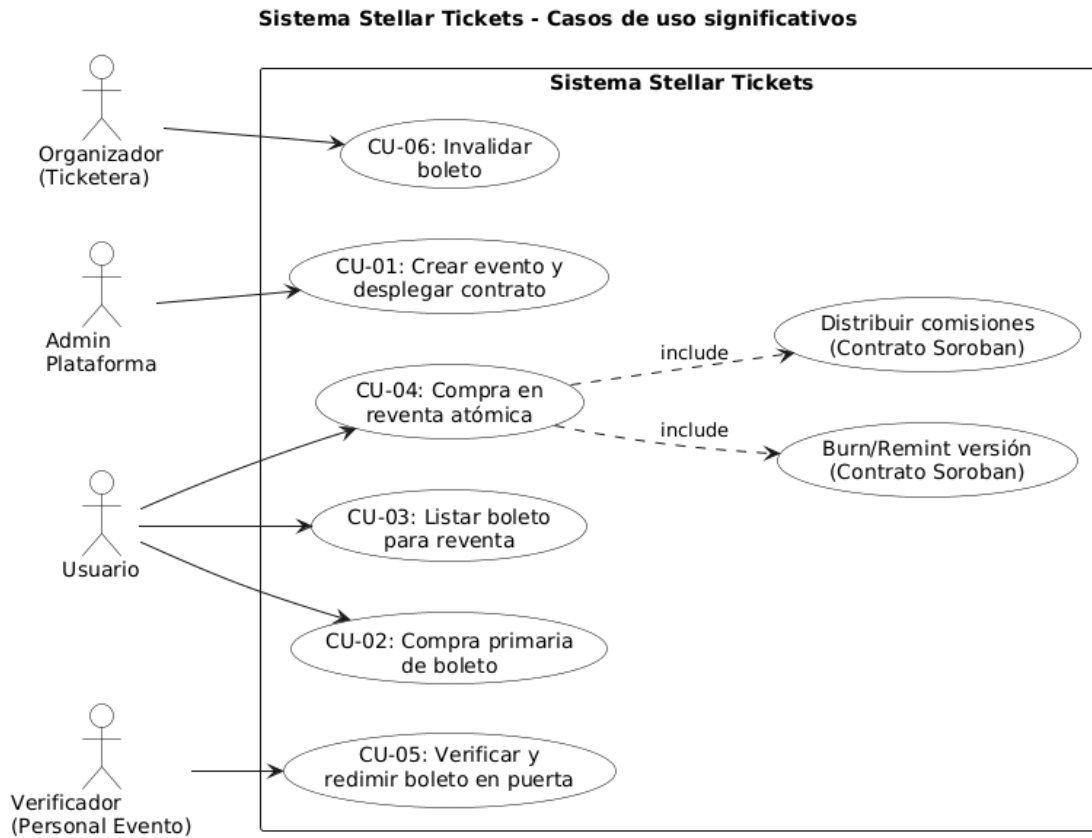


Figura 3: Diagrama de casos de uso del sistema (4 actores, 6 casos de uso principales).

El ciclo de vida funcional del boleto comprende seis fases observables on-chain: *creación* por parte del organizador; *listado* en mercado secundario; *compra* (primaria o reventa); *transferencia de propiedad* mediante *burn/remint*; *redención* por un verificador autorizado; y *cancelación administrativa* por el organizador en casos excepcionales. Las consultas de solo lectura cubren la verificación de propiedad, la versión vigente, el conjunto de boletos en reventa y el historial completo del evento.

La especificación detallada de casos de uso, los criterios de aceptación por operación y la trazabilidad entre RF, operaciones del contrato y pruebas unitarias se documentan en el anexo SRS.

5. DISEÑO DE LA SOLUCIÓN

Esta sección presenta la arquitectura del sistema y los artefactos del diseño detallado. La justificación de las tecnologías seleccionadas frente a alternativas se documenta en el anexo de Diseño y Propuesta del Trabajo de Grado.

El diagrama de contexto (Figura 4) ubica al sistema dentro de su ecosistema: usuarios finales y organizadores interactúan con SecureTicket, que a su vez se comunica con la red Stellar (Soroban RPC y Horizon) y con la billetera Freightier como agente firmante del lado del usuario.

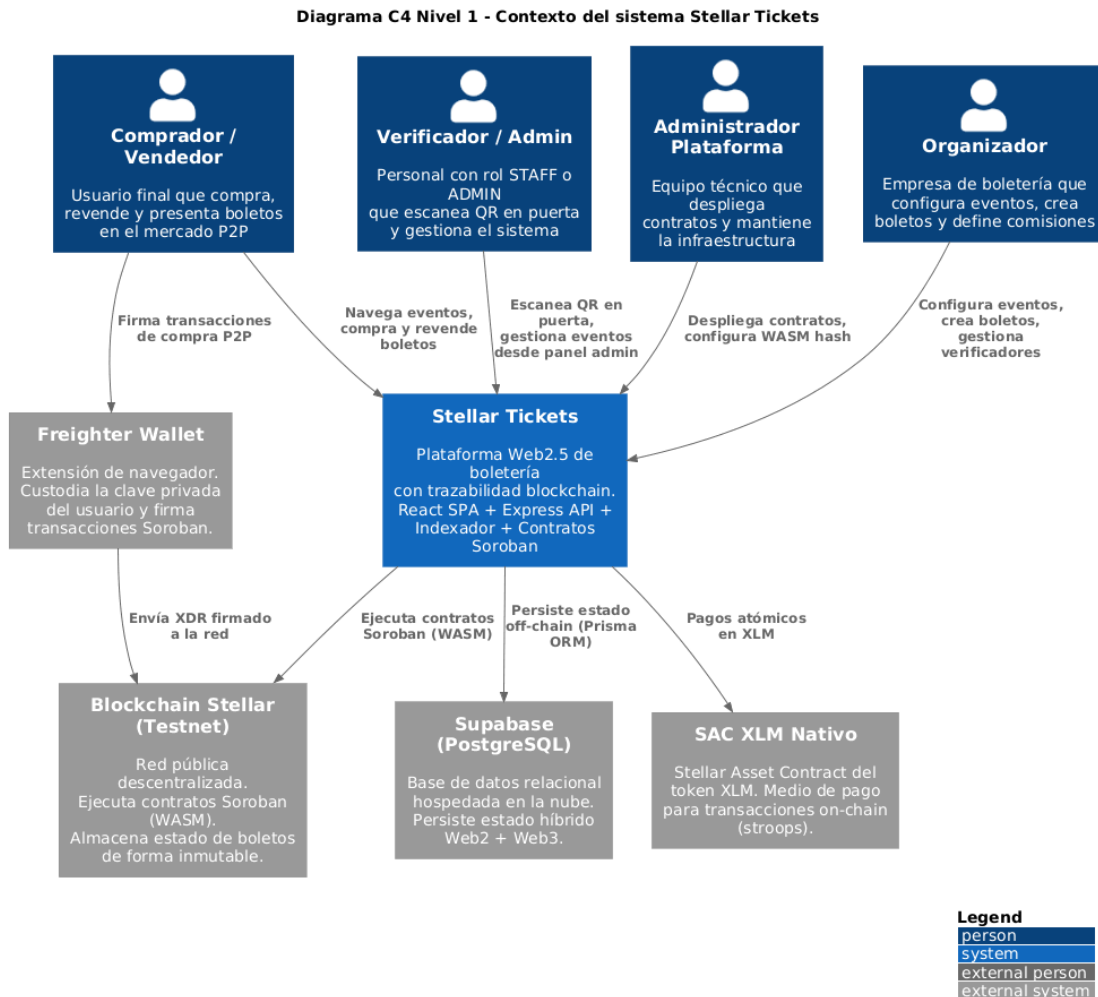


Figura 4: Diagrama de contexto C4 (nivel 1) del sistema SecureTicket.

5.1. Actores, roles y límites de confianza

El sistema separa dos planos de confianza. El plano *on-chain* reúne a los actores que firman operaciones del contrato (organizador, propietario, comprador, verificador, plataforma y administrador del *factory*); todas sus interacciones son validadas por el SCP y registradas en

el ledger. El plano *off-chain* reúne al backend, al indexador y a la base de datos relacional. El backend opera bajo un modelo de *proxy XDR stateless*: construye transacciones sin firmar cuando el firmante es el comprador, y firma con la llave del organizador únicamente las operaciones que corresponden a su rol. En ningún momento custodia llaves privadas de usuarios finales.

5.2. Modelo del ticket NFT

Cada boleto se modela dentro del contrato de evento mediante la estructura **Boleto**, cuyos campos se resumen a continuación:

- **ticket_root_id** (tipo **u32**): identificador estable del boleto a lo largo de todas sus versiones.
- **version** (tipo **u32**): contador que se incrementa con cada operación de reventa.
- **id_evento**: identificador del evento al que pertenece el boleto.
- **propietario** (tipo **Address**): dirección Stellar del dueño actual.
- **precio** (tipo **i128**): valor en la unidad mínima de XLM para aritmética entera exacta.
- Campos de estado booleano: *en_venta*, *es_reventa*, *usado* e *invalidado*.

La unicidad se garantiza por la clave de almacenamiento compuesta **Boleto(ticket_root_id, version)** y por un contador monotónicamente creciente que asigna los identificadores raíz. La autenticidad se valida mediante la firma del organizador al emitir y por la firma del propietario en cada operación posterior. La trazabilidad se materializa mediante eventos on-chain persistidos por el indexador.

5.2.1. Identificador y separación de datos sensibles

El contrato almacena exclusivamente datos operacionales públicos (identificadores, propietario, precio y estados booleanos). Los datos personales del comprador permanecen off-chain en el esquema relacional de PostgreSQL, asociados por el identificador interno del usuario y nunca por la llave privada. La vinculación entre ambas capas se realiza a través de la dirección Stellar que el usuario asocia voluntariamente al conectar la extensión Freightier en su navegador.

5.2.2. Estados y patrón burn/remint

El ciclo de vida del boleto se modela mediante tres estados:

- **ACTIVE**: el boleto ha sido emitido y es válido para el ingreso. Corresponde a la condición en la que ambos indicadores (*usado* e *invalidado*) se encuentran en falso.
- **USED**: un verificador autorizado ha redimido el boleto en puerta mediante la operación correspondiente del contrato.
- **CANCELLED**: el organizador ha invalidado el boleto de forma administrativa por circunstancias excepcionales.

En cada operación de reventa se aplica el patrón *burn/remint*: la versión vigente se marca como invalidada y se inserta una nueva entrada con versión incrementada, asignando al comprador como propietario. Un índice de versión actual se actualiza de forma atómica para que las consultas apunten siempre a la versión válida. La Figura 5 ilustra las transiciones permitidas entre los tres estados.

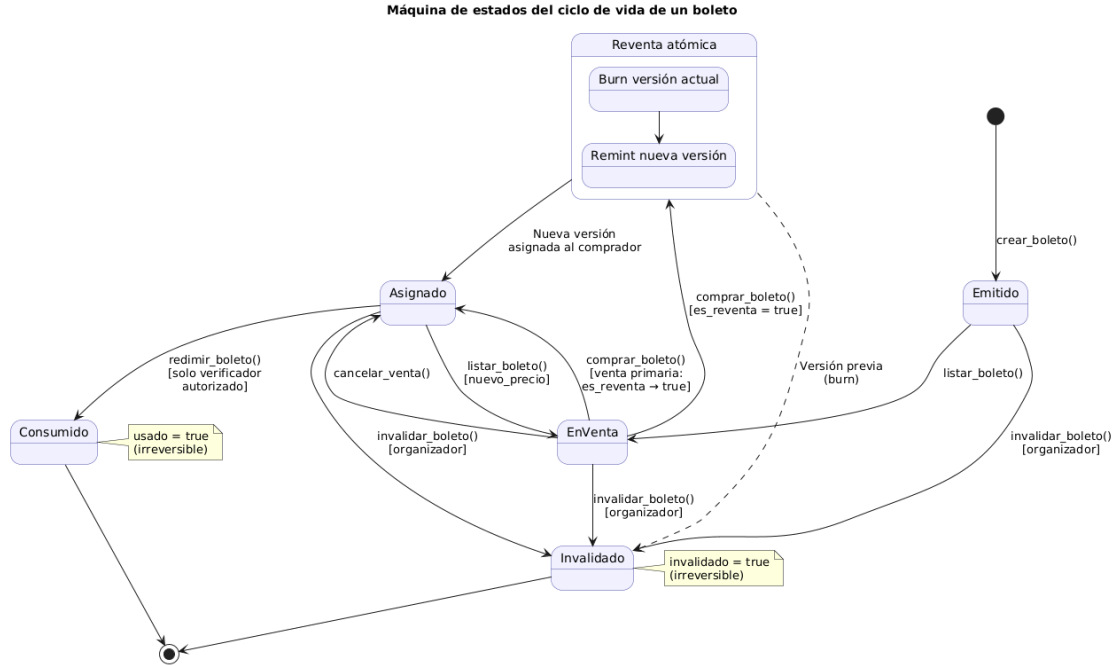


Figura 5: Diagrama de estados del boleto: ACTIVE, USED y CANCELLED, con sus transiciones.

5.3. Diseño de contratos inteligentes

El sistema utiliza tres contratos Soroban con responsabilidades acotadas: `factory_contract`, `event_contract` y `ticket_nft_contract`. Esta separación responde al principio de aislamiento de fallos: el estado de cada evento queda confinado a su propio par de contratos y la lógica de emisión NFT se desacopla de las reglas comerciales.

5.3.1. Contrato Factory

El contrato `factory_contract` implementa el patrón *Factory* para desplegar una instancia independiente del contrato de evento por cada evento creado en la plataforma. Su estructura de configuración agrupa los parámetros necesarios para la inicialización:

- Identificador del evento, dirección del organizador y token de pago.
- Porcentajes de comisión (organizador y plataforma) y sus respectivas direcciones de cobro.
- Capacidad total del aforo.

La función de creación requiere doble autenticación (administrador del *factory* y organizador) y emite el evento *EventoCreado* al completarse. Internamente, el despliegue utiliza la primitiva del entorno Soroban que permite derivar un nuevo contrato a partir del hash WASM registrado, aplicando un *salt* determinístico para garantizar direcciones reproducibles.

5.3.2. Contrato de Evento

El contrato `event_contract` expone el conjunto funcional principal del sistema. Las funciones utilizan identificadores en español como decisión de trazabilidad académica. La Tabla 6 resume las operaciones disponibles, clasificadas por tipo.

Tabla 6: Operaciones del contrato de evento.

Tipo	Operación	Descripción
Inicialización	<code>inicializar</code>	Configura el contrato con los parámetros del evento.
Emisión	<code>crear_boleto</code>	Acuña un nuevo boleto NFT para un comprador.
Reventa	<code>listar_boleto</code>	Publica un boleto en el mercado secundario.
Reventa	<code>cancelar_venta</code>	Retira el boleto del mercado secundario.
Reventa	<code>comprar_boleto</code>	Ejecuta la compra atómica (primaria o reventa).
Verificación	<code>agregar_verificador</code>	Registra un verificador autorizado en puerta.
Verificación	<code>remover_verificador</code>	Revoca la autorización de un verificador.
Redención	<code>redimir_boleto</code>	Marca el boleto como usado en el ingreso al evento.
Administración	<code>invalidar_boleto</code>	Cancela un boleto por decisión del organizador.
Consultas	<i>(conjunto de solo lectura)</i>	Estado, versión, listados y historial del evento.

La operación de compra distingue dos flujos según el estado del boleto:

- **Venta primaria:** el 100 % del precio se transfiere al organizador y el boleto queda habilitado para futuras reventas.
- **Reventa:** el contrato calcula tres montos sobre el precio listado — comisión del organizador (5 %), comisión de plataforma (3 %) y monto neto del vendedor (92 %) — y ejecuta las tres transferencias junto con el cambio de propiedad dentro de la misma invocación transaccional.

La aritmética de porcentajes utiliza una base entera de 100 (tipo `i128`) para evitar pérdida de precisión por truncamiento. Las comisiones se validan en la inicialización del contrato para que su suma no supere el 100 %.

5.3.3. Contrato NFT

El contrato `ticket_nft_contract` proporciona la representación SEP-41-compatible [25] del boleto como activo no fungible en un contrato dedicado (aproximadamente 12 KB de WASM, 10 pruebas unitarias). Expone tres operaciones principales:

- *Mint*: acuña un nuevo NFT vinculado a un propietario, un identificador raíz y una URL de metadatos.
- *Transfer*: transfiere la propiedad del NFT entre dos direcciones.
- *Burn*: destruye el NFT asociado a un identificador específico.

Cada evento despliega su propia instancia de este contrato, en correspondencia uno a uno con su contrato de evento. El identificador del token NFT se renueva en cada reventa para que las referencias almacenadas en la billetera Freightier del propietario anterior no sirvan como prueba válida en puerta.

5.3.4. Errores tipados y eventos on-chain

El contrato define dieciséis códigos de error tipados y siete eventos on-chain que el indexador consume para sincronizar la base de datos relacional. Las Tablas 7 y 8 documentan ambos conjuntos.

Tabla 7: Códigos de error del contrato de evento.

Cód.	Identificador	Condición de disparo
1	YaInicializado	Intento de reinicializar un contrato activo.
2	ComisionesMuyAltas	La suma de comisiones supera el 100 %.
3	ComisionesNegativas	Algún porcentaje de comisión es negativo.
4	PrecioInvalido	El precio del boleto es cero o negativo.
5	YaEnVenta	Boleto ya publicado en el mercado secundario.
6	BoletoUsado	Intento de operar sobre un boleto ya redimido.
7	NoEnVenta	Compra sobre un boleto no listado.
8	AutoCompra	El propietario intenta comprar su propio boleto.
9	YaUsado	Redención duplicada del mismo boleto.
10	BoletoNoEncontrado	Identificador o versión inexistente.
11	NoAutorizado	Firma no corresponde al rol requerido.
12	VersionInvalida	Versión solicitada no coincide con la vigente.
13	BoletoInvalidado	Operación sobre un boleto ya cancelado.
14	NoInicializado	Contrato invocado antes de su inicialización.
15	VerificadorYaExiste	Registro duplicado de un verificador.
16	VerificadorNoEncontrado	Revocación de un verificador no registrado.

Tabla 8: Eventos on-chain emitidos por el contrato de evento.

Evento	Operación que lo emite
boleto_creado	Acuñación de un nuevo boleto por el organizador.
boleto_listado	Publicación del boleto en el mercado secundario.
venta_cancelada	Retiro del boleto del mercado secundario.
boleto_comprado_primario	Compra en venta primaria completada.
boleto_revendido	Compra en reventa atómica completada.
boleto_redimido	Redención del boleto en puerta por verificador.
boleto_invalidado_evt	Cancelación administrativa por el organizador.

5.4. Arquitectura distribuida (tres nodos)

El prototipo se despliega en tres nodos lógicos con responsabilidades diferenciadas, lo que permite escalar de forma independiente cada plano y aislar fallos por dominio. La Figura 6 muestra el diagrama C4 de contenedores con los flujos de datos entre los nodos web, API y datos, y su conexión con la red Stellar.

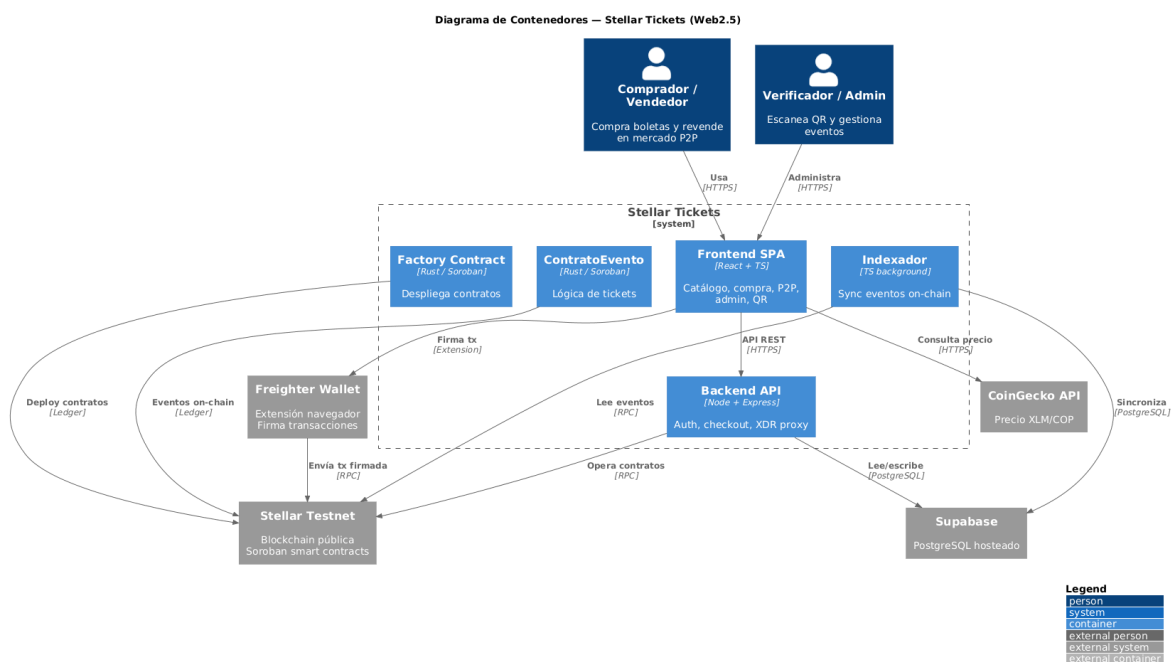


Figura 6: Diagrama C4 de contenedores (nivel 2): nodos web, API y datos, y su acoplamiento con Soroban RPC y Horizon.

5.4.1. Nodo web (frontend)

Aplicación React 18 con Vite, TypeScript, Tailwind CSS y la familia de componentes shadcn/ui (sobre Radix). La integración Web3 se apoya en dos bibliotecas: `@stellar/freighter-api` pa-

ra la firma del comprador y `@stellar/stellar-sdk` para la construcción local de transacciones de validación. El frontend expone 23 rutas que cubren el flujo de e-commerce (catálogo, carrito, *checkout*, mis entradas y ventas P2P) y los flujos administrativos (panel de gestión y escáner QR).

5.4.2. Nodo API (backend)

Backend Node.js con Express y TypeScript, sobre Prisma como ORM contra PostgreSQL. El módulo de autenticación utiliza hashing con factor de costo 10 y tokens JWT con vigencia de 7 días. La integración con Stellar se realiza mediante el SDK oficial en su versión fijada. El backend expone tres familias de endpoints:

- **Catálogo público:** consultas de eventos y boletos disponibles, con caché en memoria (TTL de 30 a 60 segundos).
- **Autenticación y checkout:** registro, inicio de sesión y flujo de compra primaria con liquidación fiat simulada.
- **Integración Soroban:** construcción de transacciones XDR para asegurar boletos, listar en reventa, cancelar listados, generar la transacción de compra para firma del comprador y enviarla a la red.

5.4.3. Nodo datos (PostgreSQL + indexador)

La base de datos relacional se hospeda en Supabase bajo el esquema `ticketing`. El modelo de datos comprende las entidades del dominio Web2 (usuarios, eventos, organizadores, recintos, tipos de boleto, inventario de asientos, carritos, órdenes y pagos) y los campos Web3 añadidos a la tabla de boletos: dirección del contrato, identificador raíz, versión, indicador de venta, precio de reventa (tipo entero largo en la unidad mínima de XLM), dirección del propietario actual y código del activo. La unicidad on-chain se refleja en un índice único compuesto por la dirección del contrato, el identificador raíz y la versión.

El indexador es un proceso de larga duración que consulta el RPC cada 5 segundos. Mantiene un cursor persistente con el último ledger procesado, consume los siete eventos del contrato con semántica idempotente y reposiciona el cursor automáticamente si cae fuera de la ventana de retención del RPC público.

5.5. Coleccionable Stellar Classic

Como capa visual adicional, el sistema emite un activo Stellar Classic asociado a cada boleto. El código del activo se compone del prefijo T seguido de 11 caracteres hexadecimales derivados del UUID del boleto. El emisor es la dirección del organizador, configurada con los indicadores de autorización revocable y recuperación habilitada (*clawback*). El flujo opera en cuatro pasos: establecimiento de línea de confianza, envío a la red, emisión idempotente del activo y transferencia mediante recuperación y pago. Esta capa complementa la representación visual del boleto en la billetera del usuario, pero no sustituye al modelo Soroban como fuente de verdad del sistema.

6. DESARROLLO DE LA SOLUCIÓN

Esta sección describe el proceso seguido para materializar el diseño en un prototipo operativo sobre *testnet*. La metodología combinó cuatro fases articuladas: revisión técnica, diseño de arquitectura, implementación incremental y evaluación experimental, ejecutadas bajo un esquema ágil con tablero Kanban y revisiones semanales con el director del trabajo de grado. El detalle de la planeación se documenta en el anexo SPMP.

6.1. Implementación de contratos Soroban

Los contratos se desarrollan en Rust contra la versión 23 de **soroban-sdk**, organizados como un Cargo *workspace* con tres paquetes correspondientes a los tres contratos del diseño. El perfil de compilación en modo *release* aplica optimización por tamaño (`opt-level="z"`), *Link-Time Optimization*, eliminación de símbolos y finalización por aborto, lo que produce módulos WASM compactos compatibles con el objetivo de compilación de Soroban. La Tabla 9 resume las métricas de cada contrato.

Tabla 9: Métricas de código fuente y pruebas por contrato.

Contrato	Líneas de código	Líneas de pruebas	Casos unitarios
Contrato de evento	≈ 1 142	≈ 709	35
Contrato <i>factory</i>	≈ 431	≈ 245	13
Contrato NFT (SEP-41)	≈ 12 KB WASM	—	10
Total			58

Las 58 pruebas se ejecutan de forma automatizada mediante el comando estándar de Cargo sobre el *workspace* completo.

6.1.1. NFT del boleto: emisión, propiedad y versionado

El boleto se modela bajo la estructura **Boleto** y una clave de almacenamiento compuesta por la tupla (*ticket_root_id*, *version*). La asignación del identificador raíz se realiza en la operación de creación mediante un contador que se incrementa atómicamente por cada emisión. Cada cambio de propiedad en reventa incrementa la versión y actualiza el puntero de versión actual, de modo que las consultas apunten siempre al ejemplar vigente del boleto.

6.1.2. Pagos atómicos vía SAC

El activo de pago utilizado es XLM, expuesto como token Soroban a través del *Stellar Asset Contract* (SAC). La transferencia se realiza mediante la invocación al método de transferencia del SAC dentro del mismo marco de llamada de la operación de compra, lo que garantiza que el pago, la distribución de comisiones y el cambio de propiedad ocupen una única transacción del ledger. Si alguna de las sub-operaciones falla, la transacción se revierte en su totalidad sin efectos parciales.

La Figura 7 muestra el diagrama de secuencia del flujo crítico de compra en reventa atómica.

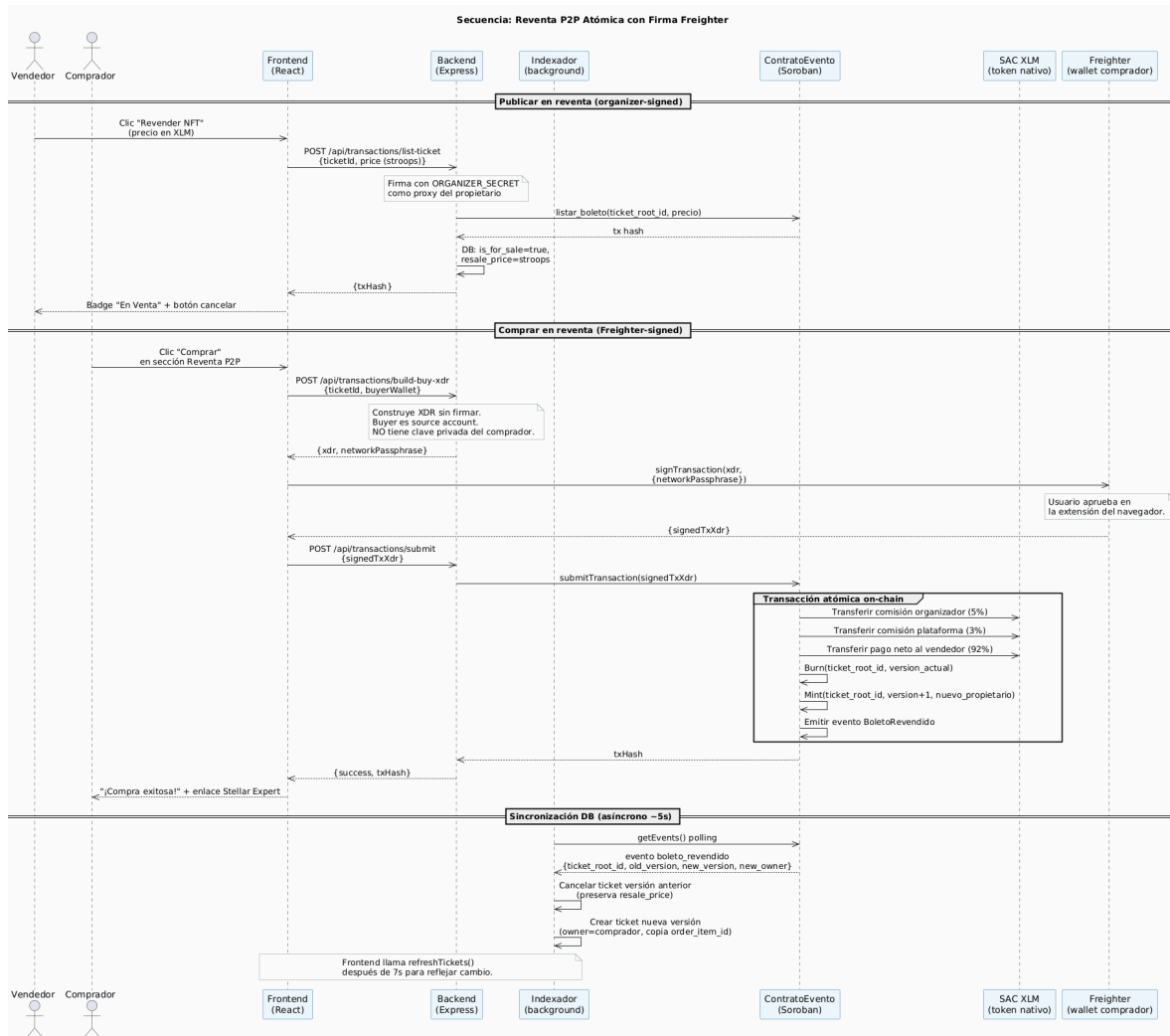


Figura 7: Diagrama de secuencia: compra en reventa atómica (pago al vendedor, comisión a organizador y plataforma, *burn* de la versión vigente y *remint* de la nueva, todo en una sola transacción).

6.1.3. Redención y control de acceso

La redención se controla mediante el rol de verificador. El organizador registra las direcciones autorizadas para validar boletos en puerta y puede revocarlas cuando lo requiera. La operación de redención valida que el invocante pertenezca al conjunto de verificadores, marca el boleto como usado y emite el evento on-chain correspondiente.

En el prototipo, el flujo en puerta opera contra la base de datos relacional para minimizar la latencia frente al usuario final, y la invocación on-chain se reserva como respaldo de auditoría. La Figura 8 resume la secuencia completa del proceso de verificación.

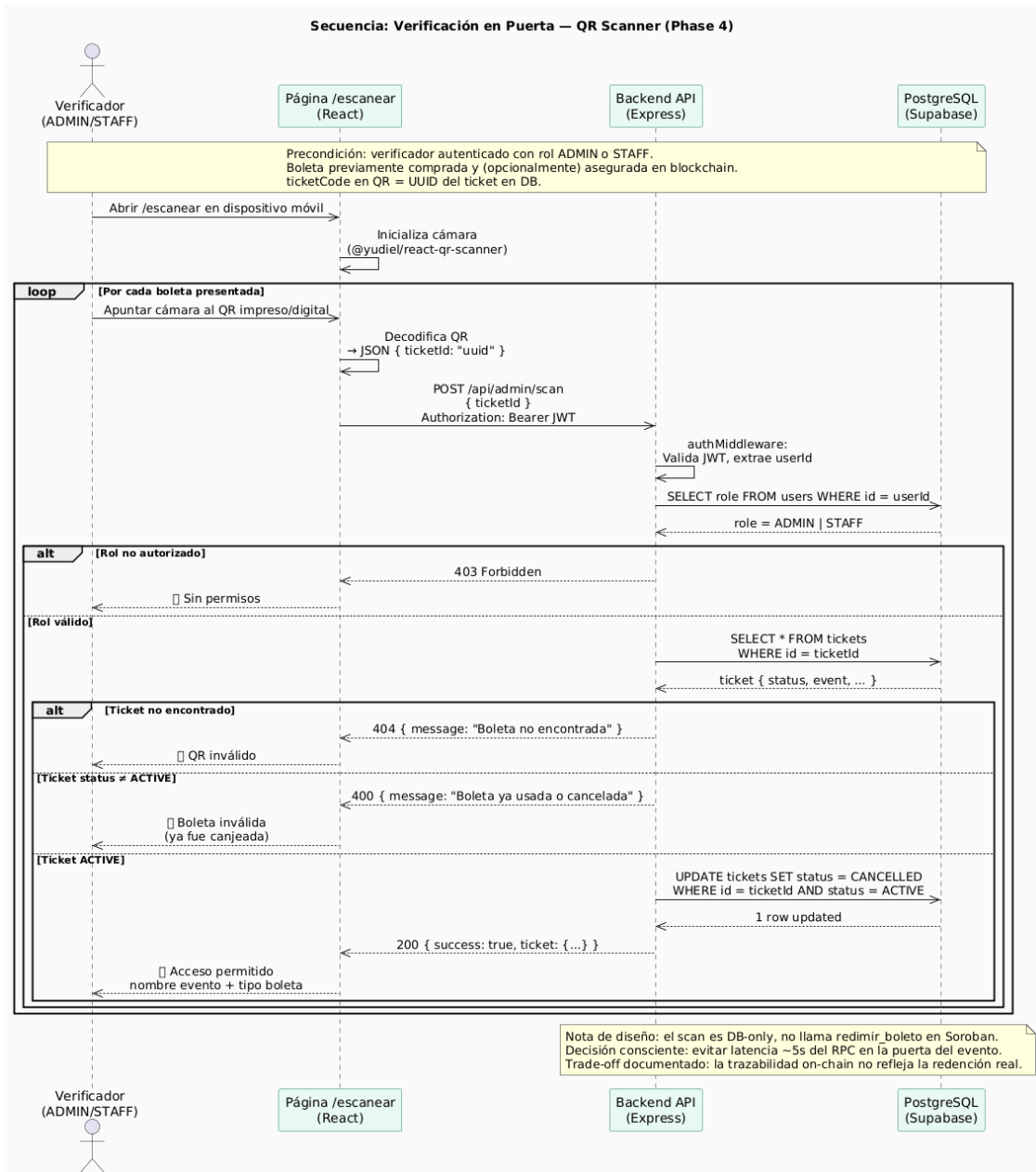


Figura 8: Diagrama de secuencia: verificación del boleto en puerta (escaneo QR, validación en PostgreSQL y respaldo on-chain).

6.2. Backend Node.js + Stellar SDK

El backend está implementado en Node.js con Express, TypeScript y Prisma [21] como ORM. La Tabla 10 resume las dependencias principales y su propósito dentro de la arquitectura.

Tabla 10: Dependencias principales del backend.

Dependencia	Propósito
<code>express + cors</code>	Servidor HTTP y control de origen cruzado.
<code>@prisma/client</code>	ORM con tipado estático contra PostgreSQL.
<code>@stellar/stellar-sdk</code> (14.4.1)	Construcción y envío de transacciones Soroban y Classic.
<code>bcryptjs</code>	Hashing de contraseñas con factor de costo configurable.
<code>jsonwebtoken</code>	Emisión y verificación de tokens JWT de sesión.
<code>dotenv</code>	Carga de variables de entorno desde archivo local.

La configuración del entorno requiere siete variables: la cadena de conexión a la base de datos, las URLs del RPC de Soroban y de Horizon, el puerto del servidor, la clave secreta para firmar los JWT, la clave secreta del organizador (para firmar operaciones on-chain de su rol) y el identificador del contrato *factory*.

6.2.1. API: usuarios, eventos, carrito, checkout y Soroban

La API expone seis familias de endpoints, organizadas por responsabilidad funcional:

- **Catálogo público:** consultas de eventos y boletos disponibles, con caché en memoria cuyo tiempo de expiración oscila entre 30 y 60 segundos según el recurso.
- **Autenticación:** registro de usuarios con hashing de contraseñas y emisión de JWT para las sesiones subsiguientes.
- **Carrito:** gestión del carrito de compras con aislamiento por usuario, garantizando que las operaciones concurrentes de distintos compradores no interfieran entre sí.
- **Checkout fiat:** flujo de compra primaria transaccional. La creación de la orden, los ítems, los boletos y el registro de pago se ejecutan dentro de una transacción Prisma atómica, de modo que una falla en cualquier paso no deje registros huérfanos.
- **Integración Soroban:** construcción de transacciones XDR para los flujos Web3: asegurar un boleto en la blockchain, listar para reventa, cancelar un listado, generar la transacción de compra para firma del comprador y enviar la transacción firmada a la red.
- **Administración:** panel del operador para la gestión de eventos, despliegue de contratos y validación de boletos en puerta mediante escaneo QR.

6.2.2. Indexador: ingesta de eventos on-chain

El indexador consulta el RPC de Soroban cada 5 segundos y mantiene un cursor persistente con el último ledger procesado (valor inicial: 100 ledgers atrás del estado actual de la red). El procesamiento se realiza en bloques de hasta 1000 ledgers por solicitud, agrupando hasta cinco direcciones de contrato por filtro para reducir el número de llamadas al RPC.

Cada uno de los siete eventos del contrato posee un manejador idempotente. En particular, el manejador del evento de reventa verifica la existencia previa del registro para evitar conflictos por reentrega, y preserva la vinculación con la orden original entre versiones sucesivas del boleto. Ante errores transitorios el indexador reintenta tras 5 segundos; si el cursor cae fuera de la ventana de retención del RPC público, el servicio analiza el mensaje de error y se reposiciona automáticamente al mínimo ledger permitido.

6.2.3. Persistencia: historial, listados y auditoría

El esquema relacional de Prisma define nueve tipos enumerados que modelan los estados del dominio: roles de usuario, estados de evento, estados del carrito, estados de la orden, métodos de pago, estados de pago, estados del inventario de asientos, estados del boleto y tipos de recinto.

Los campos Web3 añadidos al modelo de boletos — dirección del contrato, identificador raíz, versión, indicador de venta, precio de reventa, dirección del propietario y código del activo — reflejan la información on-chain necesaria para la sincronización bidireccional. La unicidad se garantiza mediante un índice único compuesto por la dirección del contrato, el identificador raíz y la versión, espejando la clave de almacenamiento del contrato Soroban.

La Figura 9 muestra el modelo entidad-relación completo de la capa off-chain.

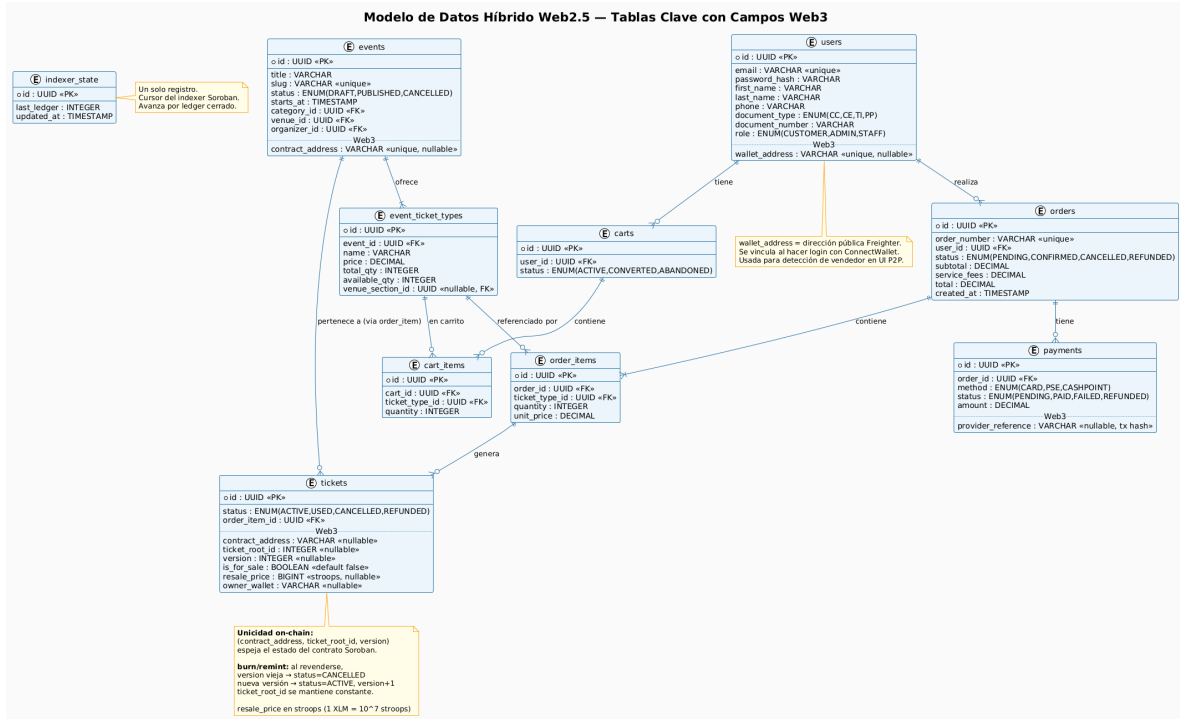


Figura 9: Modelo entidad-relación de la base de datos relacional (esquema **ticketing**).

6.3. Frontend web

La aplicación se ejecuta sobre Vite [22] y emplea TanStack Query para la gestión de caché HTTP, Framer Motion para transiciones y animaciones, y shadcn/ui [24] sobre Radix como sistema de componentes. La integración con la billetera Freightier se realiza mediante su API oficial, que expone tres funciones clave: consulta de conexión, obtención de la dirección pública y firma de transacciones serializadas en formato XDR.

El estado global de la aplicación reside en un contexto de React que comprende el usuario autenticado, el token JWT de sesión (almacenado en el almacenamiento local del navegador), el carrito, las órdenes, las entradas adquiridas, las ventas P2P activas y la dirección de la billetera vinculada. Todas las llamadas a la API se centralizan en una función utilitaria que inyecta automáticamente el encabezado de autorización.

Los componentes principales del flujo del usuario incluyen la conexión de la billetera (visible solo para usuarios autenticados), la tarjeta del boleto con las acciones disponibles según su estado (asegurar en blockchain, revender como NFT, visualizar QR) y la vista previa del precio de reventa, que muestra el valor en pesos colombianos y su equivalente estimado en XLM mediante una consulta a la API de CoinGecko [27] con caché de 5 minutos.

6.4. Despliegue distribuido

El despliegue se realiza sobre tres infraestructuras independientes, conforme al diagrama de despliegue de la Figura 10:

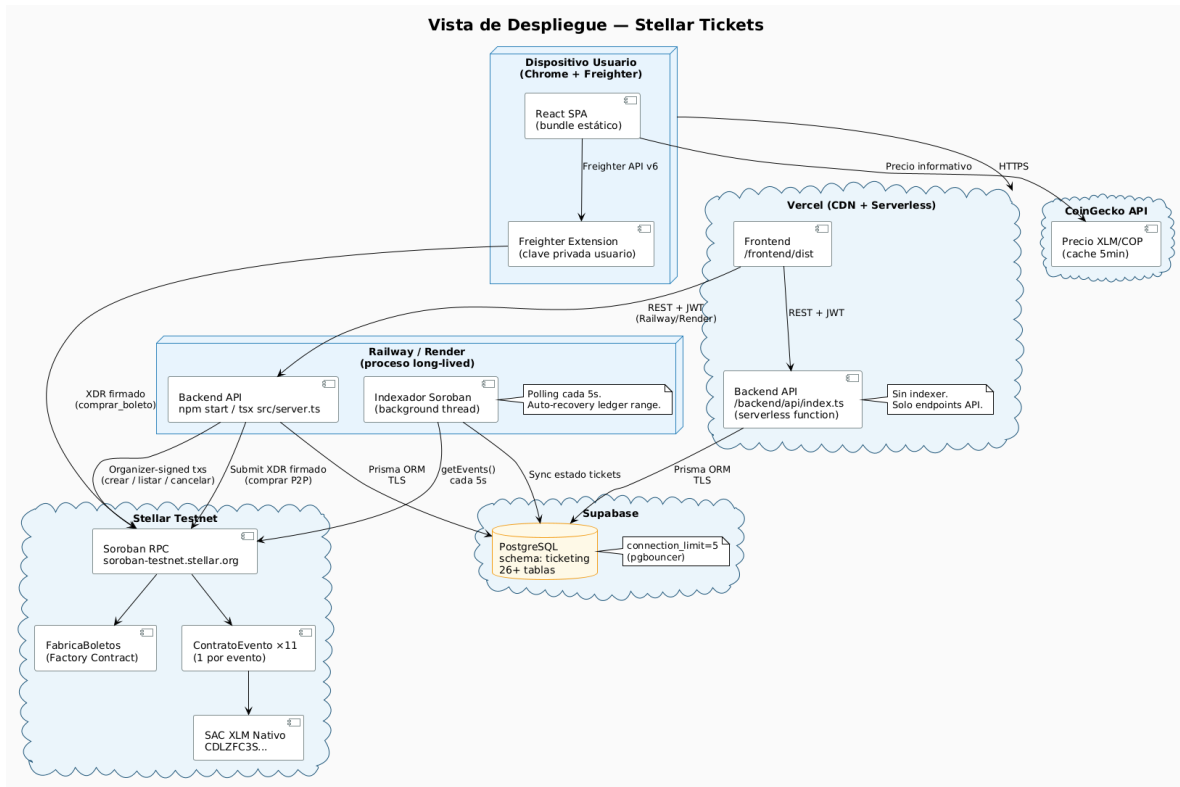


Figura 10: Diagrama de despliegue: frontend en Vercel, backend en Railway, base de datos PostgreSQL en Supabase y red Stellar testnet.

Los tres nodos físicos del prototipo se describen a continuación:

- **Frontend** en Vercel, con configuración de reescrituras para el modelo de aplicación de página única (SPA).
- **Backend** en Railway como servicio web persistente, necesario porque el indexador requiere ejecución continua en segundo plano.
- **Base de datos** PostgreSQL gestionada por Supabase, bajo el esquema `ticketing` con un límite de cinco conexiones simultáneas.

El despliegue de los contratos en la red de pruebas se ejecuta mediante scripts automatizados que realizan las siguientes operaciones en secuencia: carga de las llaves criptográficas desde las variables de entorno, financiación de las cuentas a través del servicio *friendbot* de Stellar, carga de los módulos WASM al ledger, despliegue de un par de contratos (evento y NFT) por cada evento registrado a través del *factory*, invocación de la función de inicialización con las comisiones configuradas (5 % organizador, 3 % plataforma) y persistencia de las direcciones resultantes en la base de datos. Al cierre del presente trabajo, el prototipo cuenta con doce eventos con su par de contratos desplegado.

6.5. Integración continua

El repositorio incluye un *workflow* de GitHub Actions que se ejecuta en cada *push* y *pull request* contra la rama principal. El flujo de integración realiza las siguientes tareas:

1. Instalación de la CLI de Stellar mediante descarga de binarios precompilados, sin compilación desde fuentes.
2. Compilación de cada contrato de forma individual para evitar colisiones de símbolos en el objetivo de compilación de Soroban.
3. Ejecución de la suite completa de 58 pruebas unitarias.
4. Publicación del módulo WASM compilado como artefacto del *pipeline*.

Esta automatización constituye la evidencia operacional de la trazabilidad de cambios sobre el código fuente y garantiza que toda modificación que ingrese a la rama principal haya superado la totalidad de las pruebas.

6.6. Pruebas

La estrategia de verificación se organiza en cuatro niveles complementarios:

1. **Pruebas unitarias sobre contratos:** 58 casos que cubren el ciclo de vida completo del boleto, las precondiciones de cada operación, los dieciséis códigos de error tipados, la doble autenticación del *factory* y las operaciones de emisión, transferencia y quema del NFT.
2. **Pruebas de integración E2E:** verificación del flujo completo desde la perspectiva del usuario, incluyendo registro, navegación por el catálogo, compra primaria, aseguramiento del boleto en la blockchain, listado para reventa, compra de reventa con firma desde Freight, redención en puerta y cancelación administrativa.
3. **Pruebas antifraude:** escenarios negativos diseñados para validar que el contrato rechaza operaciones ilícitas. Se verifican los casos de doble redención, doble compra concurrente, intento de autocompra del propietario, redención por un usuario no autorizado y manipulación del precio de reventa.
4. **Pruebas de medición:** captura de tiempos de respuesta y costos de gas sobre la red de pruebas, cuyos resultados alimentan la sección de Resultados y permiten contrastar el desempeño real del prototipo contra los umbrales definidos en los requerimientos no funcionales.

7. RESULTADOS

Esta sección consolida los resultados experimentales obtenidos sobre la red de prueba de Stellar, conforme al proceso de control de calidad definido en la metodología (pruebas unitarias, integración, antifraude y medición). Los resultados se reportan en cuatro frentes: desempeño del contrato, cobertura de pruebas, comportamiento antifraude y discusión comparativa.

7.1. Metodología de medición

Las métricas se capturan sobre testnet con el contrato `event_contract` desplegado por el *factory* configurado para el prototipo. Las variables observadas son: tiempo de envío al RPC, tiempo de confirmación on-chain (cierre del ledger que incluyó la transacción), latencia extremo a extremo desde la interfaz hasta la actualización en la base de datos relacional, y tasa de fallos por tipo de operación. Cada operación se ejecutó en corridas independientes y los percentiles se calculan sobre el conjunto observado.

7.2. Desempeño sobre testnet

La Tabla 11 resume la latencia y el costo nominal por tipo de operación. El costo se reporta en pesos colombianos (COP) calculados a la tasa de referencia promediada de $1 \text{ XLM} \approx 1\,500 \text{ COP}$. Esta tasa es indicativa y está sujeta a la alta variabilidad del precio del activo en el mercado; la cifra on-chain real es determinística en la unidad mínima de XLM.

Tabla 11: Latencia de confirmación y costo nominal por operación sobre testnet.

Operación	Envío (s)	Conf. p50 (s)	Conf. p95 (s)	Costo aprox. (COP)
crear_boleto	<1	4,8	6,1	$\approx 0,02$
listar_boleto	<1	4,7	5,9	$\approx 0,02$
cancelar_venta	<1	4,7	5,8	$\approx 0,02$
comprar_boleto (primaria)	<1	4,9	6,3	$\approx 0,03$
comprar_boleto (reventa)	<1	5,1	6,5	$\approx 0,06$
redimir_boleto	<1	4,7	5,9	$\approx 0,02$
invalidar_boleto	<1	4,8	6,0	$\approx 0,02$

El costo de la compra de reventa es mayor porque la transacción contiene cuatro operaciones lógicas dentro del mismo *call frame*: transferencia al vendedor (92 %), comisión al organizador (5 %), comisión a la plataforma (3 %) y actualización del registro de propiedad. La semántica atómica garantiza que las cuatro se aplican como una sola unidad o ninguna se aplica en caso de fallo.

La latencia extremo a extremo desde la interfaz incorpora la construcción del XDR en el backend, la firma del usuario en Freighter y la propagación del evento por el indexador (poll

cada 5 segundos). El tiempo p95 desde la confirmación del usuario hasta la actualización de la vista de “Mis entradas” se ubica entre 8 y 11 segundos.

7.3. Cobertura y resultados de pruebas automatizadas

La suite unitaria sobre los contratos comprende 58 casos. La Tabla 12 resume su distribución.

Tabla 12: Distribución de pruebas unitarias sobre los contratos Soroban.

Contrato	Casos	Cobertura funcional
<code>event_contract</code>	35	Ciclo de vida, autorización por rol, errores tipados, eventos.
<code>factory_contract</code>	13	Doble autenticación, despliegue por <code>salt</code> , eventos.
<code>ticket_nft_contract</code>	10	Emisión, transferencia, quema y consultas SEP-41.
Total	58	Ejecutadas en GitHub Actions en cada <i>push</i> y <i>pull request</i> .

Los 58 casos se ejecutan de forma exitosa en el *workflow* de integración continua. La cobertura es del 100 % sobre los siete eventos definidos por el `event_contract` y sobre los 16 códigos de error tipados.

7.4. Evaluación antifraude

Los escenarios antifraude se modelaron como pruebas negativas. La Tabla 13 consolida los seis escenarios y el resultado observado.

Tabla 13: Escenarios antifraude evaluados sobre el contrato.

Escenario	Resultado esperado	Error / Mecanismo
Doble redención del mismo boleto	Rechazo en la 2ª invocación	YaUsado (código 9)
Doble compra del mismo boleto listado	Rechazo en la 2ª compra	NoEnVenta (código 7) tras <i>burn</i>
Compra del propio boleto (autocompra)	Rechazo	AutoCompra (código 8)
Redención por dirección no autorizada	Rechazo	NoAutorizado (código 11)
Manipulación del precio en el <i>frontend</i>	Firma inválida	Validación on-chain del precio listado
Coexistencia de dos copias válidas tras reventa	Imposible por diseño	Patrón <i>burn/remint</i> sobre la tupla (<code>root_id, version</code>)

En los seis escenarios el contrato rechazó la operación inválida sin dejar estados parciales y

emitió el error tipado correspondiente. La trazabilidad de los intentos fallidos queda registrada en el meta del ledger y es consultable mediante *Stellar Expert* [26].

7.5. Análisis y discusión

Los resultados se contrastan con dos referencias: el esquema tradicional de reventa (QR estático y conciliación bancaria) y los esquemas NFT sobre Ethereum 1.0.

Frente al esquema tradicional, la solución reduce el tiempo de finalidad de 72–120 horas hábiles al rango de 3–5 segundos del cierre de ledger en Stellar. El costo nominal por operación se desplaza de un esquema porcentual (1,5–3,5 % más tarifa fija) a una tarifa fija del orden de centésimas de peso colombiano por operación, lo que vuelve económicamente viable la operación con boletos de bajo precio. La duplicidad por QR estático queda eliminada por construcción.

Frente a los esquemas sobre Ethereum 1.0, la capacidad teórica de la red Stellar (1 000–3 000 TPS) representa una diferencia del orden de 100x sobre el rango característico de 15–30 TPS [28], lo que permite considerar el modelo para eventos de alta concurrencia. La semántica atómica de hasta 100 operaciones por transacción habilita la compra de reventa como una unidad indivisible sin requerir patrones de *commit-reveal* o *escrow* externos.

Las limitaciones identificadas son: (i) la evaluación se realiza sobre testnet, lo que implica condiciones de carga distintas a las de mainnet bajo demanda real; (ii) la latencia E2E está dominada por el intervalo de *poll* del indexador (5 s), parámetro configurable a costa de mayor consumo del RPC; (iii) el costo nominal corresponde al *base fee*: en saturación del ledger, el *surge pricing* puede incrementar el costo efectivo, comportamiento que no se observó durante la evaluación.

7.6. Evidencia funcional del prototipo

A continuación se incluyen capturas del prototipo desplegado en producción de pruebas (`secure-ticket-beta.vercel.app`) que evidencian los flujos descritos en las secciones anteriores.

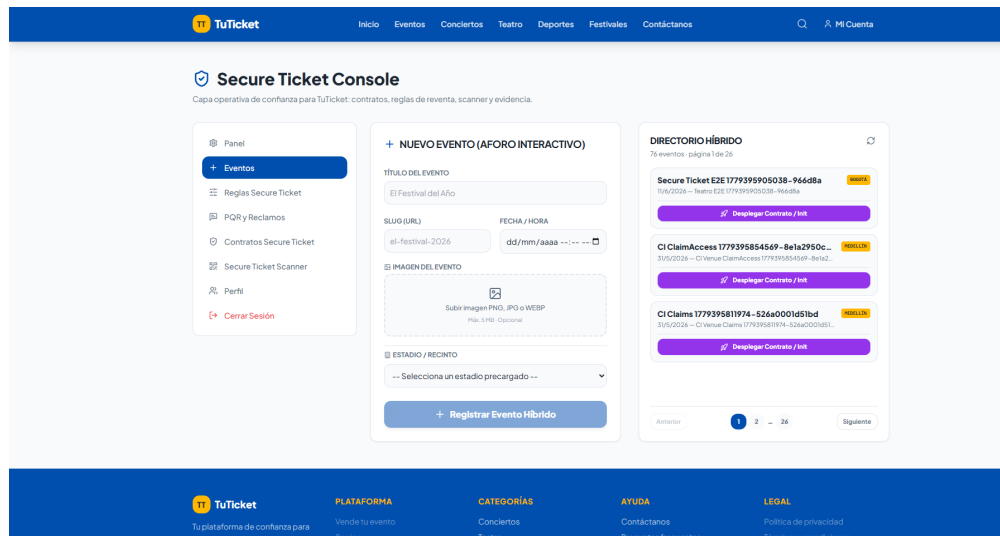


Figura 11: Consola del usuario administrador: panel desde el que se despliegan los contratos Soroban para eventos *offchain* y se crean los eventos, vinculando cada evento de la pasarela tradicional con su contrato *onchain* correspondiente a través del **factory**.

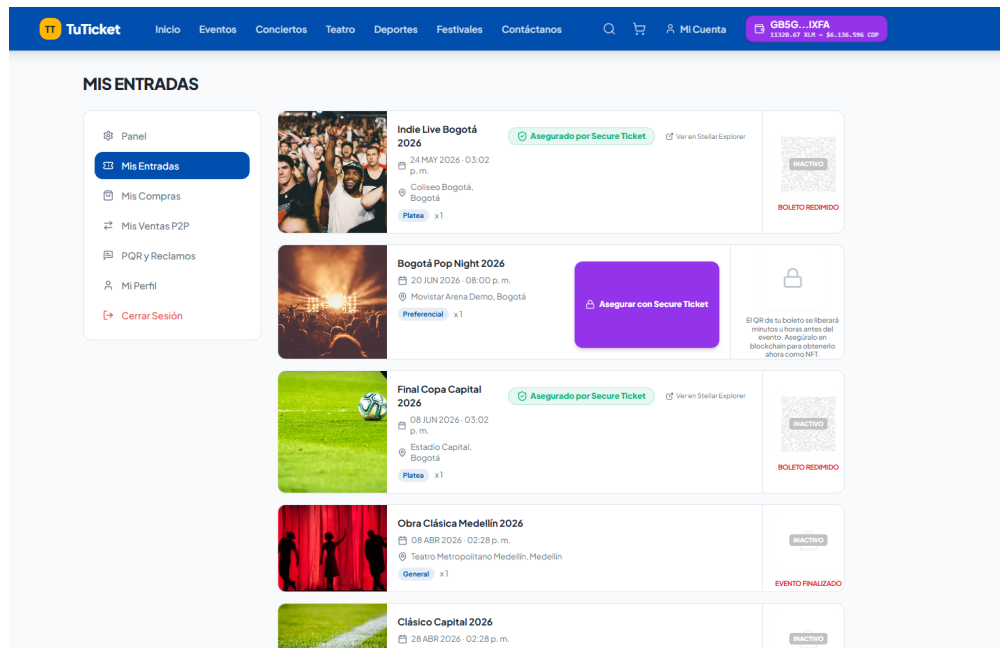


Figura 12: Vista “Mis Entradas” del usuario final: los boletos que aún no han sido asegurados en la blockchain se manejan de forma tradicional, por lo que el usuario debe esperar a que el organizador libere el QR de acceso; el botón “Asegurar en Blockchain” permite acuñar el boleto como NFT y liberar el QR de inmediato.

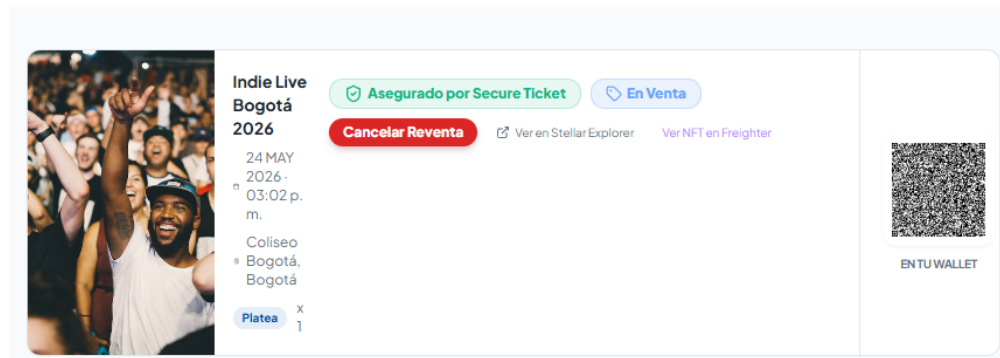


Figura 13: Boleto ya asegurado en la blockchain: el QR firmado queda liberado de forma anticipada y la interfaz expone las opciones disponibles sobre el NFT (revender en el mercado P2P, guardar el QR como *collectible* en la wallet, ver el detalle del contrato).

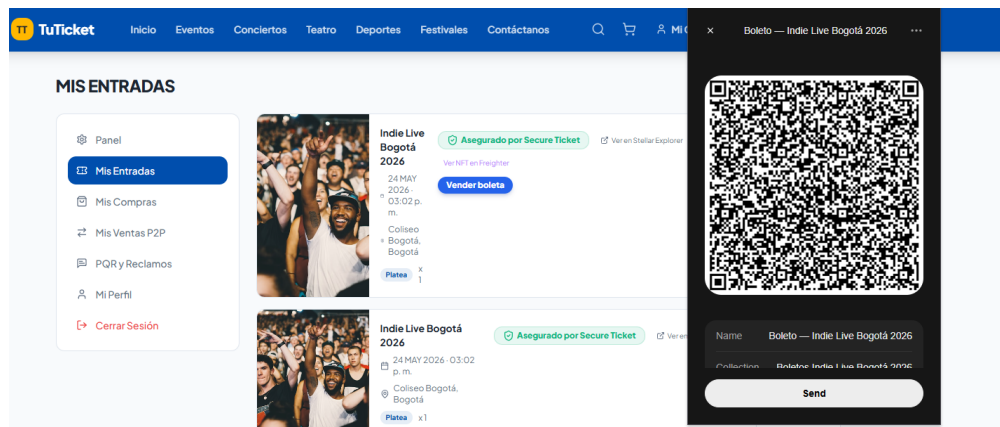


Figura 14: Una vez asegurado el boleto, el usuario puede guardar su QR como NFT *collectible* dentro de la wallet Freightier; la captura muestra cómo se visualiza el NFT del boleto dentro de la extensión.

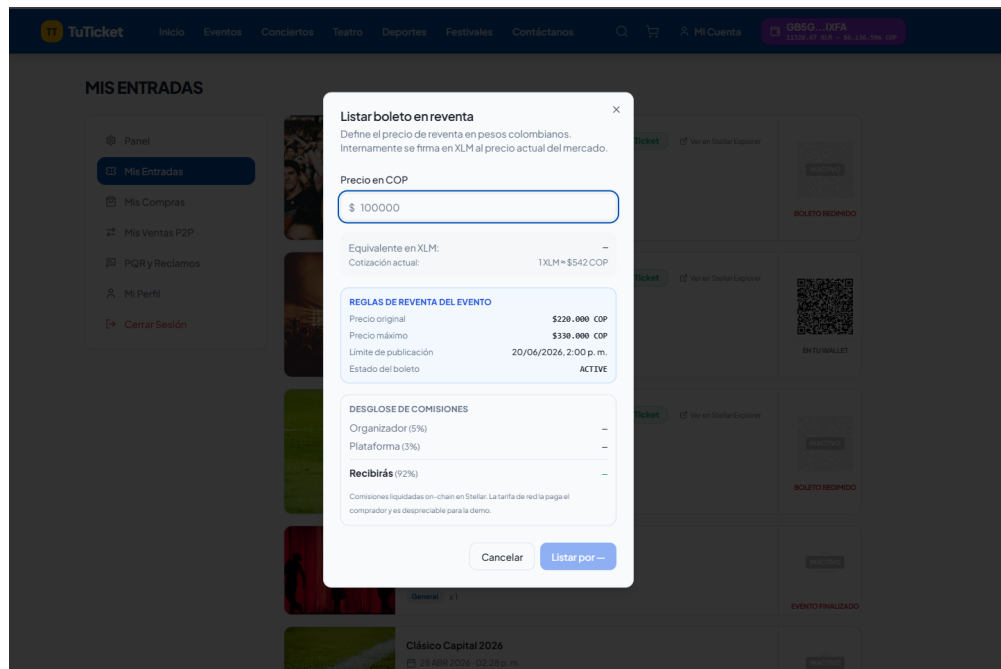


Figura 15: Listado de un boleto para reventa: el usuario fija el precio en COP, la interfaz muestra el equivalente en XLM a la cotización actual y descompone las comisiones (5 % organizador, 3 % plataforma) junto al monto neto a recibir.

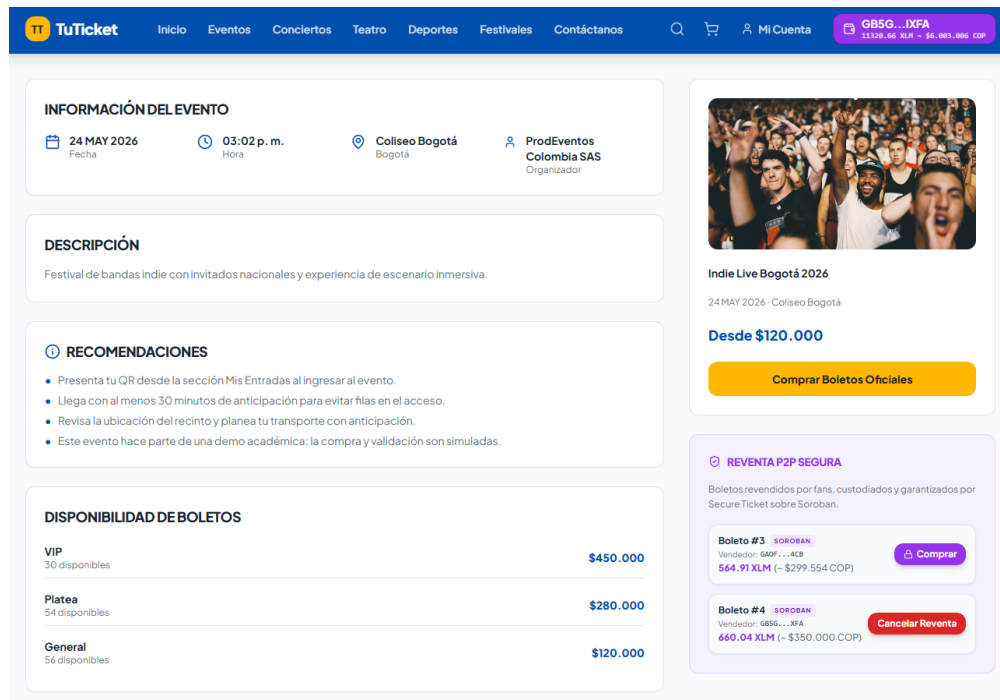


Figura 16: Vista pública del mercado secundario: así ve un usuario una boleta publicada en la red Stellar disponible para compra P2P, con el precio fijado por el revendedor, el detalle del evento y el botón para ejecutar la compra atómica contra el contrato.

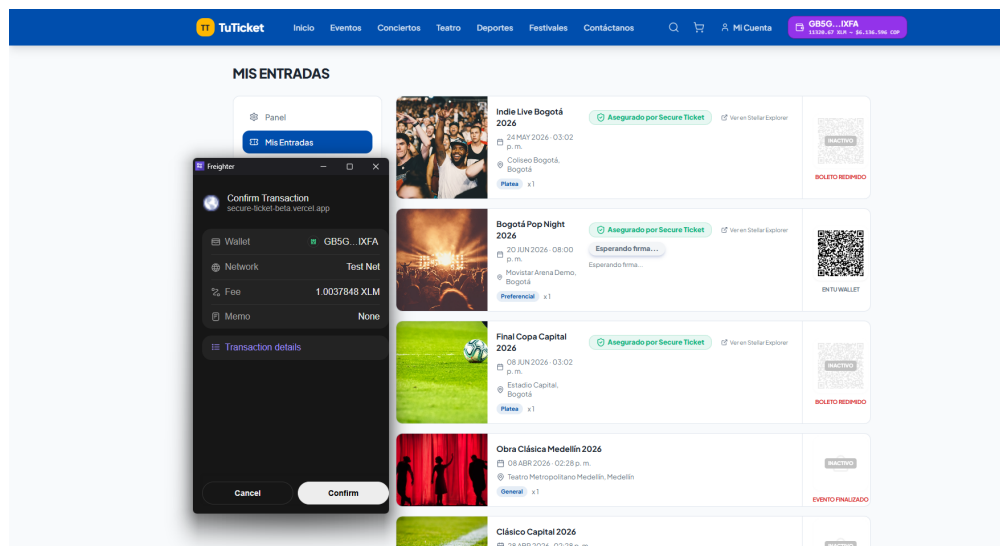


Figura 17: Ejemplo de cómo el usuario ve en la extensión Freighter el detalle de la transacción Soroban construida por el backend (XDR proxy) y cómo esta solicita su firma con la llave privada local; esta confirmación se solicita tanto al publicar un boleto en reventa como al momento de comprarlo, y las llaves privadas nunca abandonan la extensión.

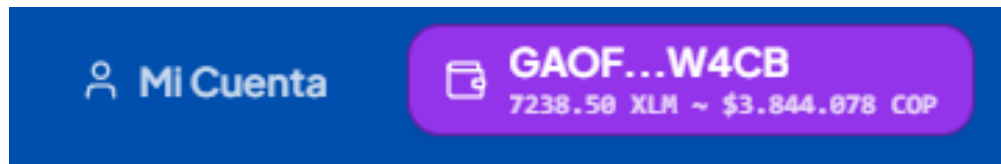


Figura 18: Detalle de la wallet del usuario una vez vinculada su sesión con la extensión Freighter: la interfaz muestra el saldo en XLM y su equivalente en COP a la cotización actual, integrando la información on-chain de la cuenta con la representación monetaria local.

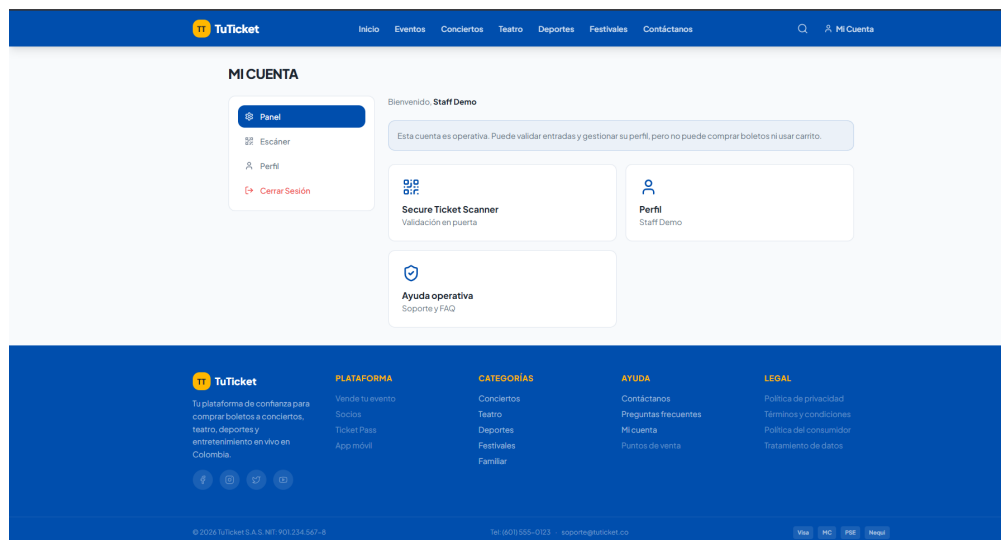


Figura 19: Consola del personal de *staff* (puerta de acceso): los validadores en puerta acceden a un panel reducido cuya única funcionalidad disponible es el “Secure Ticket Scanner”, utilizado para escanear y validar los QR NFT presentados por los asistentes.

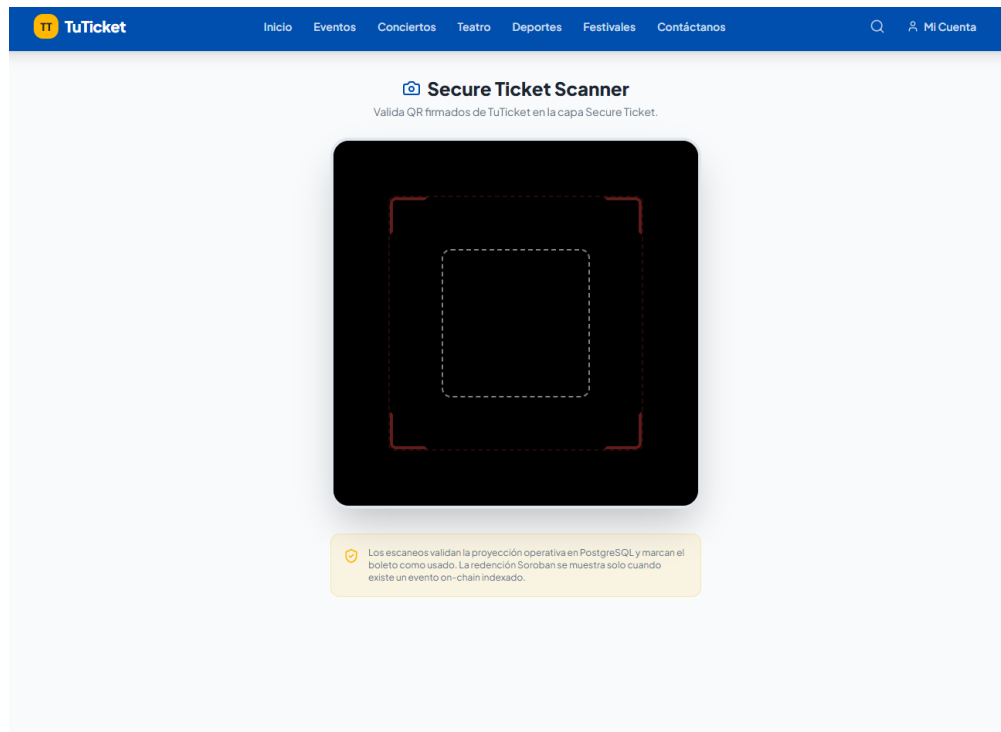


Figura 20: Vista del *Secure Ticket Scanner* tal como la observa el personal de *staff* en puerta: la cámara escanea el QR NFT del asistente y la interfaz reporta el resultado de la verificación contra el contrato `ticket_nft_contract`, marcando el boleto como redimido en caso de éxito.

8. CONCLUSIONES

8.1. Análisis de Impacto del Proyecto

8.1.1. Impacto en ingeniería de sistemas

El trabajo aporta tres elementos verificables sobre el código fuente público del prototipo. En primer lugar, un modelo de boleto NFT con versionado y patrón *burn/remint* que elimina por construcción la coexistencia de dos copias válidas del mismo activo, problema característico de los esquemas con QR estático. En segundo lugar, una arquitectura de tres nodos con un patrón *XDR proxy stateless* en el backend, que garantiza que las llaves privadas de los usuarios finales nunca abandonen su entorno local: el comprador firma desde Freightier, mientras que el organizador firma server-side solo las operaciones que le competen por rol. En tercer lugar, un conjunto de mecanismos verificables de seguridad por construcción (autorización por rol on-chain, atomicidad de operaciones combinadas y trazabilidad pública del historial de propiedad) con evidencia localizable en el repositorio y en las pruebas automatizadas.

A corto plazo, el prototipo proporciona una base de referencia para grupos académicos interesados en patrones de *ticketing* con contratos inteligentes sobre Stellar, particularmente en lo relativo al diseño de operaciones atómicas combinadas. A mediano plazo, la separación entre la fuente de verdad on-chain y la capa relacional indexada off-chain habilita la incorporación de analítica, gobernanza de precios y detección de patrones de especulación sin alterar el contrato. A largo plazo, la arquitectura es extensible a otros dominios donde la unicidad y la trazabilidad pública sean propiedades requeridas (membresías, certificaciones, activos coleccionables).

8.1.2. Impacto global, económico, ambiental y social

En el plano **económico**, la reducción del costo nominal por operación de un esquema porcentual a una tarifa fija del orden de centésimas de peso colombiano ($\approx 0,02$ – $0,06$ COP a la tasa de referencia) hace económicamente viable la reventa de boletos de bajo precio, segmento desatendido por las plataformas tradicionales debido a las comisiones mínimas absolutas de las pasarelas bancarias. La distribución automática de comisiones reduce la fricción operativa entre vendedor, organizador y plataforma.

En el plano **ambiental**, el SCP de Stellar no utiliza minería ni *Proof of Work*: el consenso se alcanza mediante intercambio de mensajes firmados entre nodos, lo que elimina el costo energético asociado al cómputo intensivo característico de redes basadas en PoW. Esta propiedad es relevante en el contexto de la creciente preocupación por la huella energética de las tecnologías de registro distribuido.

En el plano **social**, la trazabilidad pública del historial de propiedad fortalece la posición del comprador final frente a la reventa fraudulenta y abre la posibilidad de mecanismos de cumplimiento automatizado de obligaciones parafiscales como la prevista en la Ley 1493 de 2011 colombiana. El modelo no-custodio del backend reduce la superficie de exposición ante incidentes de seguridad sobre el operador de la plataforma.

A escala **global**, la propuesta inscribe el trabajo en una línea de investigación activa sobre aplicaciones B2B de tecnologías de registro distribuido en sectores con problemas de fraude y opacidad, sin requerir migración total del usuario final hacia esquemas Web3 puros: el flujo de

e-commerce tradicional se conserva y la capa on-chain opera como complemento opt-in.

8.2. Conclusiones y Trabajo Futuro

Los cuatro objetivos específicos planteados al inicio del trabajo se cumplieron. La materialización de cada boleto como NFT identificado por (`ticket_root_id`, `version`), el contrato `event_contract` con transferencia atómica de propiedad y distribución automática de comisiones, el prototipo desplegado sobre testnet con doce eventos independientes (cada uno con su par `event_contract` + `ticket_nft_contract`) y la aplicación web con flujo E2E completo constituyen evidencias verificables del cumplimiento, todas localizables en el repositorio público y en las pruebas automatizadas que se ejecutan en GitHub Actions.

La hipótesis técnica del trabajo, según la cual una solución basada en contratos inteligentes Soroban sobre Stellar es viable como sustento de un sistema B2B de reventa de entradas con garantías por construcción de unicidad, autenticidad y trazabilidad, queda respaldada por los resultados experimentales. La latencia de confirmación medida (4,7–5,1 s en mediana), el costo nominal (del orden de 0,02–0,06 COP por operación a la tasa de referencia) y la cobertura de los 58 casos de prueba automatizados son consistentes con los parámetros declarados en la documentación oficial de la red y suficientes para soportar el caso de uso académico evaluado.

Las extensiones identificadas como trabajo futuro se agrupan en cuatro líneas. **Despliegue productivo**: migración controlada a *mainnet* con una pasarela de pago fiat real, procesos de identidad KYC y procedimientos de respuesta a incidentes. **Políticas de mercado avanzadas**: tope dinámico de reventa, devoluciones parciales, listas de bloqueo de direcciones y soporte para activos de pago adicionales (USDC vía SAC). **Interoperabilidad**: definición de una interfaz B2B documentada para integración con plataformas comerciales de *ticketing*. **Analítica**: evaluación de patrones de detección de especulación y modelos de scoring de riesgo a partir del historial indexado en PostgreSQL.

Referencias

- [1] The Ticket Fairy. «How to Avoid Fake Tickets: The Rise of Secondary Market Scams,» visitado 22 de mayo de 2026. dirección: <https://www.ticketfairy.com>
- [2] DataIntel, «Global Event Ticket Resale Market Size, Share, Trends and Forecast,» DataIntel Insights, inf. téc., 2024. dirección: <https://dataintel.com>
- [3] Superintendencia de Industria y Comercio de Colombia, *Resolución de Sanción por Cartelización en la Reventa de Boletas de las Eliminatorias al Mundial Rusia 2018*, Superintendencia de Industria y Comercio, 2020. dirección: <https://www.sic.gov.co>
- [4] Stellar Development Foundation. «Soroban Smart Contracts: Getting Started,» visitado 22 de mayo de 2026. dirección: <https://developers.stellar.org/docs/build/smart-contracts/getting-started>
- [5] Stellar Development Foundation. «¿Qué son las Stablecoins y cómo funcionan?» Visitado 22 de mayo de 2026. dirección: <https://stellar.org/es/aprender/que-son-las-stablecoins>
- [6] D. Mazières, «The Stellar Consensus Protocol: A Federated Model for Internet-Level Consensus,» Stellar Development Foundation, inf. téc., 2015. dirección: <https://www.stellar.org/papers/stellar-consensus-protocol.pdf>
- [7] Stellar Development Foundation. «SEP-0041: Token Interface for Soroban Smart Contracts,» visitado 22 de mayo de 2026. dirección: <https://developers.stellar.org/docs/tokens/quickstart>
- [8] Binance Academy. «What Is Stellar (XLM)?» Visitado 22 de mayo de 2026. dirección: <https://academy.binance.com/en/articles/what-is-stellar-xlm>
- [9] Visa Inc. «Visa Fact Sheet: Global Infrastructure and Transaction Processing,» visitado 22 de mayo de 2026. dirección: <https://usa.visa.com>
- [10] S. Nakamoto, «Bitcoin: A Peer-to-Peer Electronic Cash System,» inf. téc., 2008. dirección: <https://bitcoin.org/bitcoin.pdf>
- [11] Cambridge Centre for Alternative Finance. «Cambridge Bitcoin Electricity Consumption Index (CBECI),» visitado 22 de mayo de 2026. dirección: <https://ccaf.io/cbeci>
- [12] Stellar Development Foundation, «Stellar Carbon Footprint and Sustainability Assessment Framework,» Stellar Development Foundation & PwC US, inf. téc., 2024. dirección: <https://stellar.org>
- [13] International Energy Agency, «Electricity 2024: Analysis and forecast to 2026,» International Energy Agency (IEA), inf. téc., 2024. dirección: <https://www.iea.org/reports/electricity-2024>
- [14] ISO/IEC, «ISO/IEC 25010:2011 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models,» International Organization for Standardization, inf. téc., 2011. dirección: <https://www.iso.org/standard/35733.html>
- [15] OWASP Foundation. «Transport Layer Security Cheat Sheet,» visitado 22 de mayo de 2026. dirección: https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html

- [16] Stellar Development Foundation. «Stellar Developers Documentation: APIs, SDKs and Tools,» visitado 22 de mayo de 2026. dirección: <https://developers.stellar.org/docs/data/apis>
- [17] Stellar Development Foundation. «Introducción a Stellar: Conceptos Básicos y Arquitectura,» visitado 22 de mayo de 2026. dirección: <https://stellar.org/es/aprender/intro-a-stellar>
- [18] Congreso de la República de Colombia, *Ley 1493 de 2011: Por la cual se regulan los espectáculos públicos de las artes escénicas, se reconocen formalmente los derechos de autor y se dictan otras disposiciones*, 2011. dirección: <http://secretariassenado.gov.co>
- [19] IEEE Computer Society, «IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications,» Institute of Electrical y Electronics Engineers, inf. téc., 1998. dirección: <https://standards.ieee.org/standard/830-1998.html>
- [20] Stellar Development Foundation. «Stellar Consensus Protocol: Prueba de Acuerdo,» visitado 22 de mayo de 2026. dirección: <https://stellar.org/es/aprender/stellar-consensus-protocol>
- [21] Prisma Data, Inc. «Prisma: Next-generation Node.js and TypeScript ORM,» visitado 23 de mayo de 2026. dirección: <https://www.prisma.io/>
- [22] Vite Contributors. «Vite: Next Generation Frontend Tooling,» visitado 23 de mayo de 2026. dirección: <https://vitejs.dev/>
- [23] Meta Platforms, Inc. «React: The library for web and native user interfaces,» visitado 23 de mayo de 2026. dirección: <https://react.dev/>
- [24] shadcn. «shadcn/ui: Beautifully designed components that you can copy and paste into your apps,» visitado 23 de mayo de 2026. dirección: <https://ui.shadcn.com/>
- [25] Stellar Development Foundation. «SEP-0041: Smart Contract Token Interface,» visitado 23 de mayo de 2026. dirección: <https://github.com/stellar/stellar-protocol/blob/master/ecosystem/sep-0041.md>
- [26] OrbitLens. «Stellar Expert: Network explorer and analytics platform for Stellar,» visitado 23 de mayo de 2026. dirección: <https://stellar.expert/>
- [27] CoinGecko. «CoinGecko Cryptocurrency Data API,» visitado 23 de mayo de 2026. dirección: <https://www.coingecko.com/en/api>
- [28] Ethereum Foundation. «Scaling Ethereum,» visitado 23 de mayo de 2026. dirección: <https://ethereum.org/en/developers/docs/scaling/>

A. ANEXOS

Los anexos del trabajo se entregan como documentos independientes y no forman parte del cómputo de páginas de esta memoria. Cada anexo es autocontenido y la memoria los referencia donde corresponde para evitar duplicación de contenido.

Tabla 14: Anexos del Trabajo de Grado.

Id.	Documento	Contenido principal
A	Plan de Gestión del Proyecto (SPMP)	Calendarización, presupuesto, riesgos, calidad, configuración y cierre.
B	Especificación de Requisitos (SRS)	Requerimientos funcionales y no funcionales, casos de uso, modelo de datos, criterios de aceptación.
C	Diseño (SDD) y Propuesta de TG	Vistas C4, secuencias UML, modelo on-chain/off-chain y justificación tecnológica.

Anexo A. Plan de Gestión del Proyecto (SPMP). Contiene la vista general del proyecto, los supuestos y restricciones, los entregables, la calendarización ejecutada, el presupuesto, los riesgos materializados, las prácticas de calidad, la gestión de configuración y el cierre retrospectivo del proyecto. Incluye cinco diagramas BPMN del proceso de gestión y la lista completa de hitos cumplidos hasta el cierre del semestre 2026-1.

Anexo B. Especificación de Requisitos de Software (SRS). Contiene la definición detallada de los veinte requerimientos funcionales y los veinte no funcionales del prototipo organizados por módulo, los casos de uso por actor, el modelo de datos relacional, las interfaces del backend con Soroban RPC y Horizon, los criterios de aceptación y la trazabilidad entre requerimientos, operaciones del contrato y pruebas unitarias.

Anexo C. Especificación del Diseño y Propuesta del Trabajo de Grado. Documento que consolida la propuesta original del trabajo, el documento de diseño (SDD) con las vistas C4 (contexto, contenedores, componentes y despliegue), los diagramas de secuencia de los flujos críticos (compra primaria, reventa atómica y redención) y el modelo de datos completo on-chain/off-chain. Incluye el análisis de alternativas tecnológicas y la justificación de la selección de Stellar/Soroban frente a otras plataformas de contratos inteligentes.

Material complementario. Repositorio público del proyecto: <https://github.com/Tesis-Stellar/Secure-Ticket>.